

Python Programming with Artificial Intelligence

Deep Learning

Huangxun Chen @IoT&AI Thrust



Deep Model Architecture

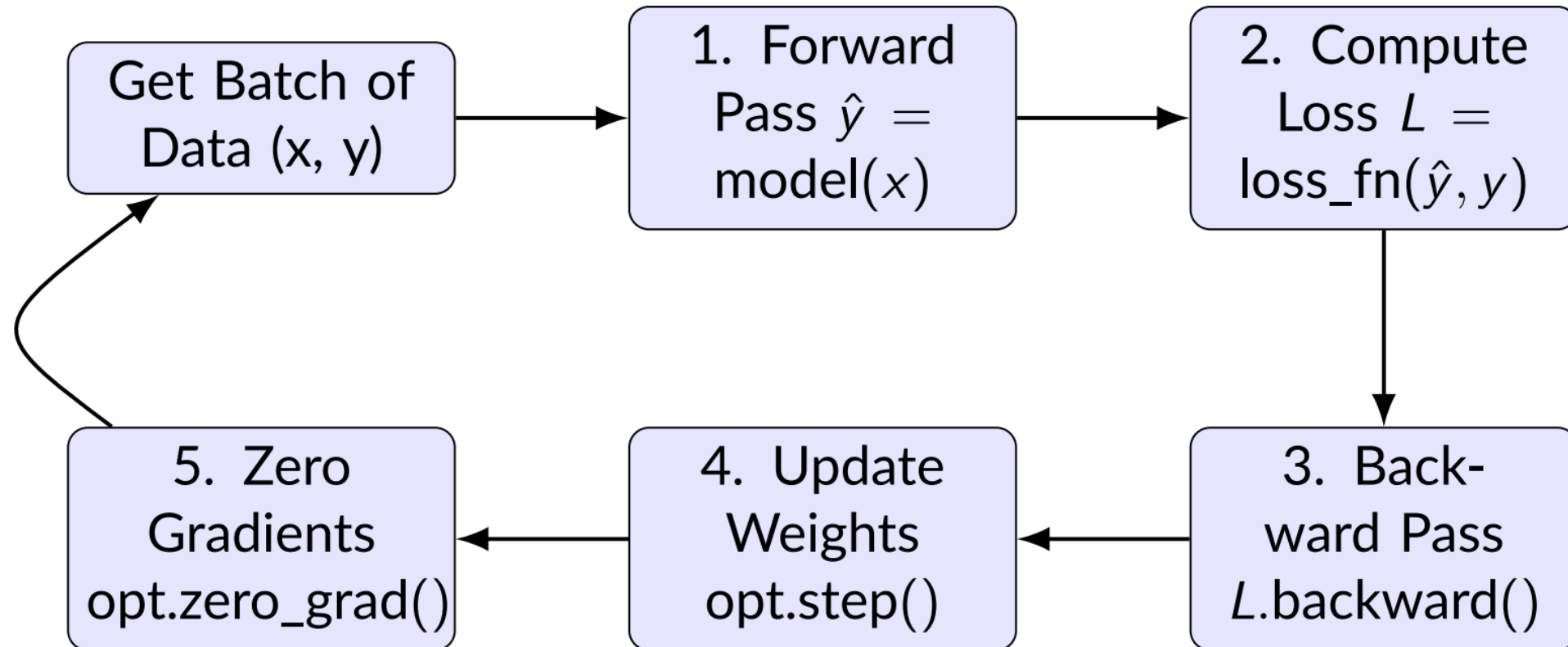
- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Sequence Modeling and RNN
- Language Modeling and Transformer

Training Strategy

- Regularization
- Normalization
- Weight Initialization & Optimization

Recap: The Training Loop

Find the "right" values for all the weights (w) and biases (b)



Two Key Questions

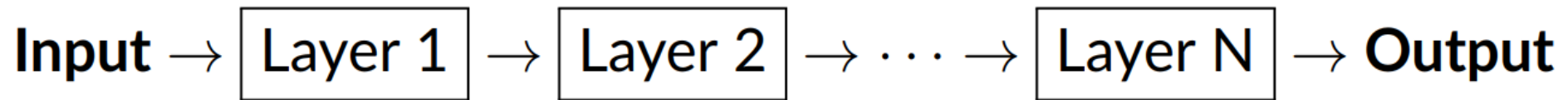
- **How do we design the model architecture?**
Choose suitable layers and organize them properly in code.
- **How do we train the model?**
Choose proper loss functions, initialization, optimizers, regularization



Model Architecture

Deep Model

"Deep" refers to the depth of layers, which allows the model to learn a hierarchy of complex concepts automatically from data.



There is **no one-size-fits-all solution** and no universal shortcut.

Choose the right tool for the right task:

- MLPs for simple structured data
- CNNs for images
- RNNs for sequential data
- Transformers for language modeling

The AI community has developed many reusable building blocks.

Understanding their design logic helps us customize models for real-world projects.

Next, we introduce several commonly used building blocks in PyTorch.

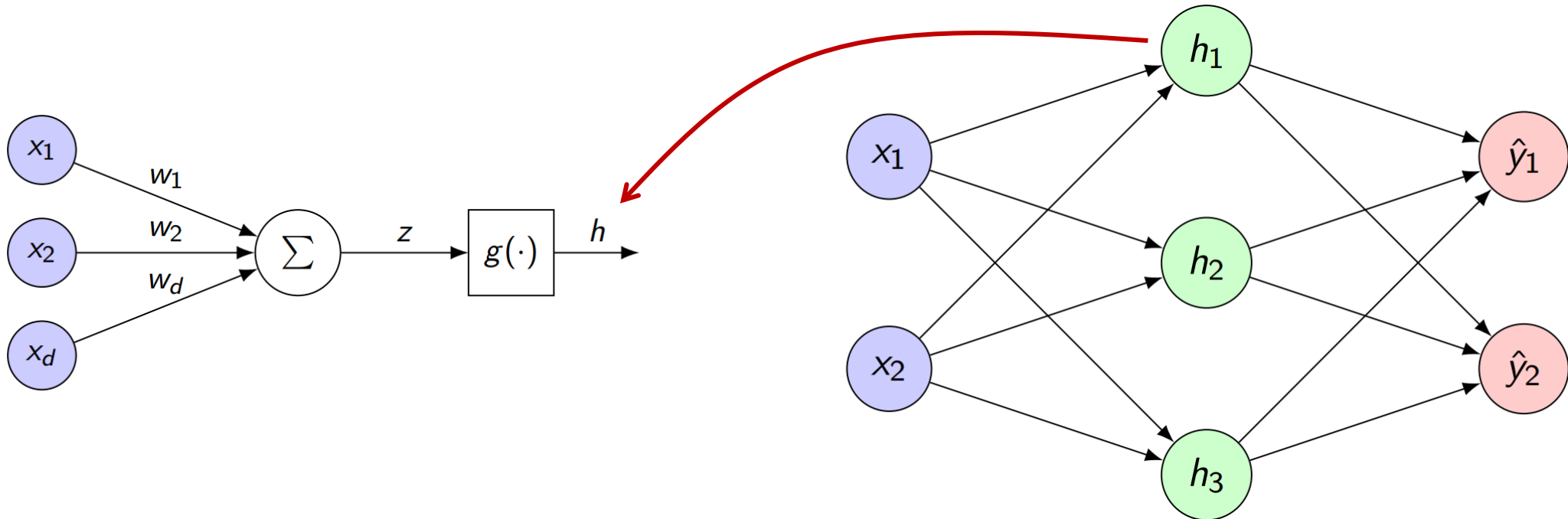
- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Sequence Modeling and RNN
- Language Modeling and Transformer



MLP

Multi-Layer Perceptron (MLP)

- A neuron computes a weighted combination of inputs and then applies a nonlinearity. It acts as a simple feature detector.
- A layer with M inputs and N outputs is a fully-connected or linear layer.
- A matrix multiplication followed by an activation.
- Input Layer $\rightarrow$ Hidden Layer(s) $\rightarrow$ Output Layer



■ The Role of Activation Functions

A stack of linear layers is **one single linear layer**.

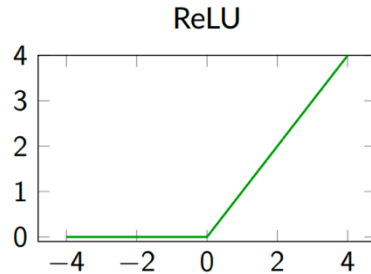
The network cannot learn complex, non-linear patterns.

We must apply a **non-linear activation function** after each linear layer.

$$\begin{array}{ll} \text{Layer 1: } y_1 = W_1x + b_1 & \text{Substitute Layer 1 into Layer 2:} \\ \text{Layer 2: } y_2 = W_2y_1 + b_2 & \xrightarrow{\hspace{1cm}} \end{array}$$
$$\begin{aligned} y_2 &= W_2(W_1x + b_1) + b_2 \\ y_2 &= (W_2W_1)x + (W_2b_1 + b_2) \\ y_2 &= W_{\text{new}}x + b_{\text{new}} \end{aligned}$$

Common Activations: Sigmoid, Tanh, ReLU

ReLU: $\max(0, x)$
the default

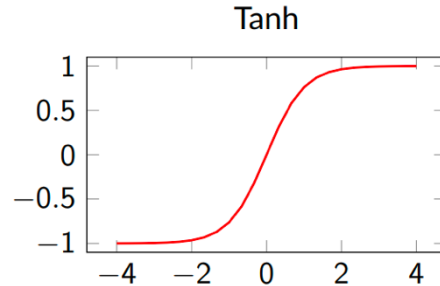


Gradients can flow "uninterrupted" through the network, no matter how deep, makes training much faster and more stable.

Good: No vanishing gradient for $x > 0$ (Slope is 1)

Bad: Dying ReLU (gradient is 0 for all $x < 0$).

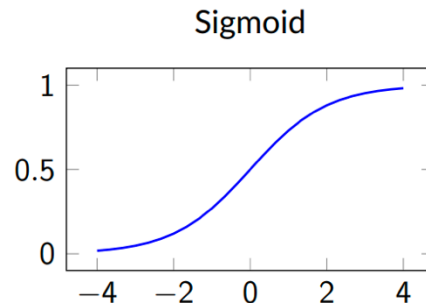
Tanh: $\tanh(x)$



Good: Zero-Centered

Bad: Vanishing gradients (gradient ≈ 0 for large/small x)

Sigmoid: $\frac{1}{1+e^{-x}}$



During backpropagation, gradients are multiplied. $0.1 \times 0.1 \times 0.1 = 0.001$. The signal to update the early layers "vanishes".

Bad: Vanishing gradients (gradient ≈ 0 for large/small x)

■ A Special Activations: Softmax

Converts a vector of scores (logits) into a probability distribution.

- Properties: All outputs are (0, 1) and all outputs sum to 1.
- Not a hidden layer activation.
- Used only for multi-class classification output layers.

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

nn.Linear in nn.Module

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        # Define the layers (weights are created automatically)
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
    def forward(self, x):
        # Define the computation flow
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        return x
model = MLP(input_dim=784, hidden_dim=128, output_dim=10)
print(model)
```



Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNN)

CNN is proposed to address MLP limitation.

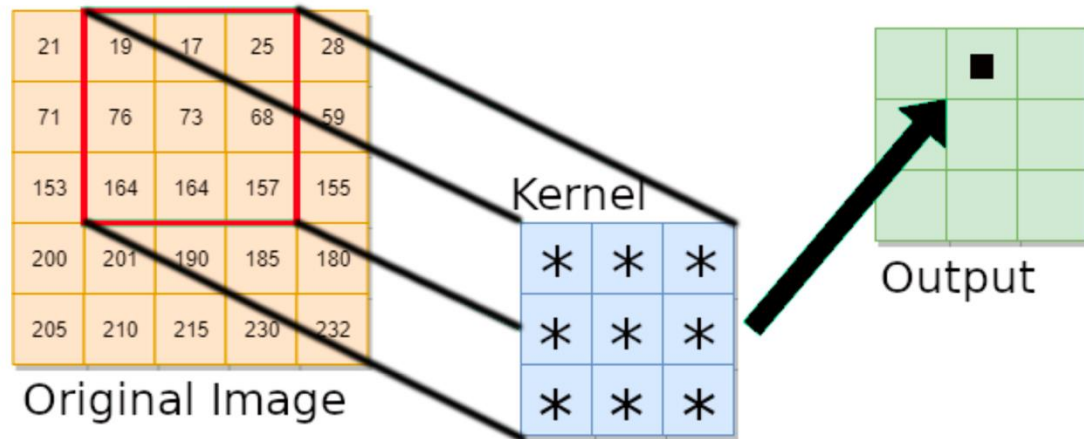
Problem: Can we just use a big MLP on an image?

Let's try on a "small" 256x256 RGB image:

- Input dimensions: $256 \times 256 \times 3 = 196,608$ features.
- If our first hidden layer has 1,000 neurons (modest)...
- Number of weights = $196,608 \times 1,000 = 196.6$ Million
- This is for one layer. The model would be enormous, slow, and would overfit instantly.

Convolutional Neural Networks (CNN)

Convolution Operation: Small filters slide over image as local feature detectors.



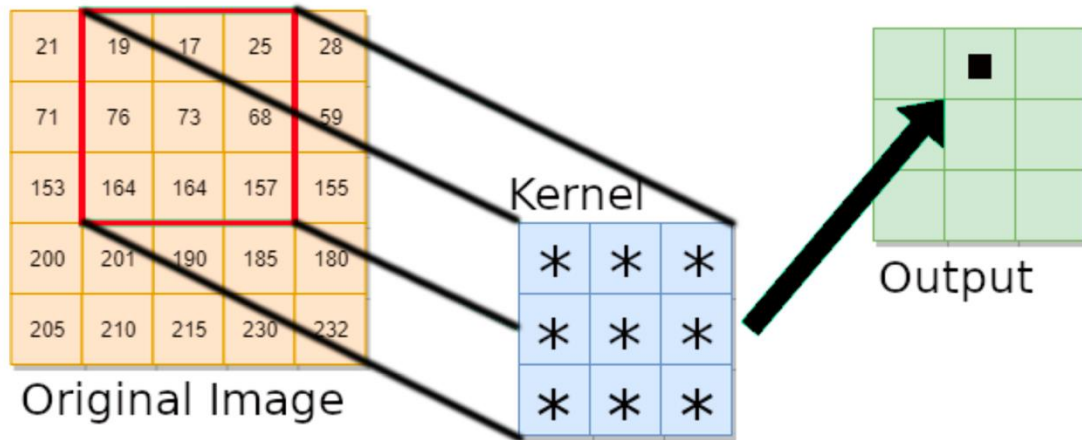
Idea: Patterns are local. Pixels near each other are related (e.g., an "eye").

Compared with MLP: An MLP treats the top-left pixel and bottom-right as equally related.

A "filter" or kernel (e.g., 3x3) slides over the input image. At each step, it computes an element-wise multiplication and sums the result (a dot product)

Convolutional Neural Networks (CNN)

Parameter Sharing: Same filter weights used everywhere.



A "filter" or kernel (e.g., 3x3) slides over the input image. At each step, it computes an element-wise multiplication and sums the result (a dot product)

Idea: Features are translation-invariant. A "cat" in the top-left corner is the same "cat" as one in the bottom-right.

Compared with MLP:

- An MLP would have to re-learn "cat" at every possible position.
- 3x3 filter needs only 9 weights vs. 196M for MLP.

A real CNN uses multiple channels.

- Input Channels (C_{in}): A color image (RGB) has 3 input channels.
- Output Channels (C_{out}): A convolution layer applies multiple filters (e.g., 64 filters) to the input.
- Each filter produces one output channel (one feature map).
- The final output is a stack of feature maps

Channels in CNN

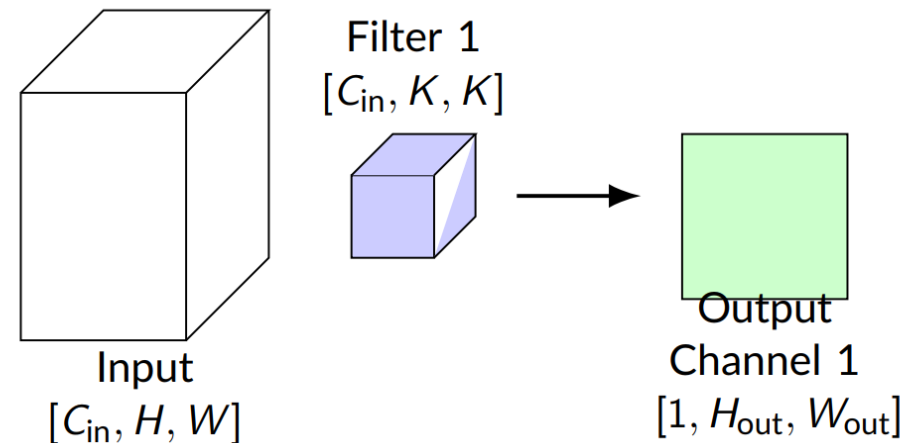
- An input with C_{in} channels (e.g., 3 for RGB) is not 2D, it's 3D.
- Therefore, **the filter must also be 3D**. It must **have the same depth as the input**.
- A "3x3 filter" on an RGB image is actually a 3x3x3 block of weights.
- This 3D filter slides over the 3D input, computes one dot product ($3 \times 3 \times 3 = 27$ multiplications), and produces one number.
- To get C_{out} (e.g., 64) output channels, we need 64 different 3x3x3 filters.

```
# 3 input channels (RGB)
# 64 output channels (64 filters)
# 3x3 kernel size
conv1 = nn.Conv2d(in_channels=3, out_channels=64,
                  kernel_size=3)
```

Input Shape: $[\text{Batch}, C_{in}, H_{in}, W_{in}]$

Filter Shape: $[C_{out}, C_{in}, K_H, K_W]$

Output Shape: $[\text{Batch}, C_{out}, H_{out}, W_{out}]$



Stride and Padding in CNN

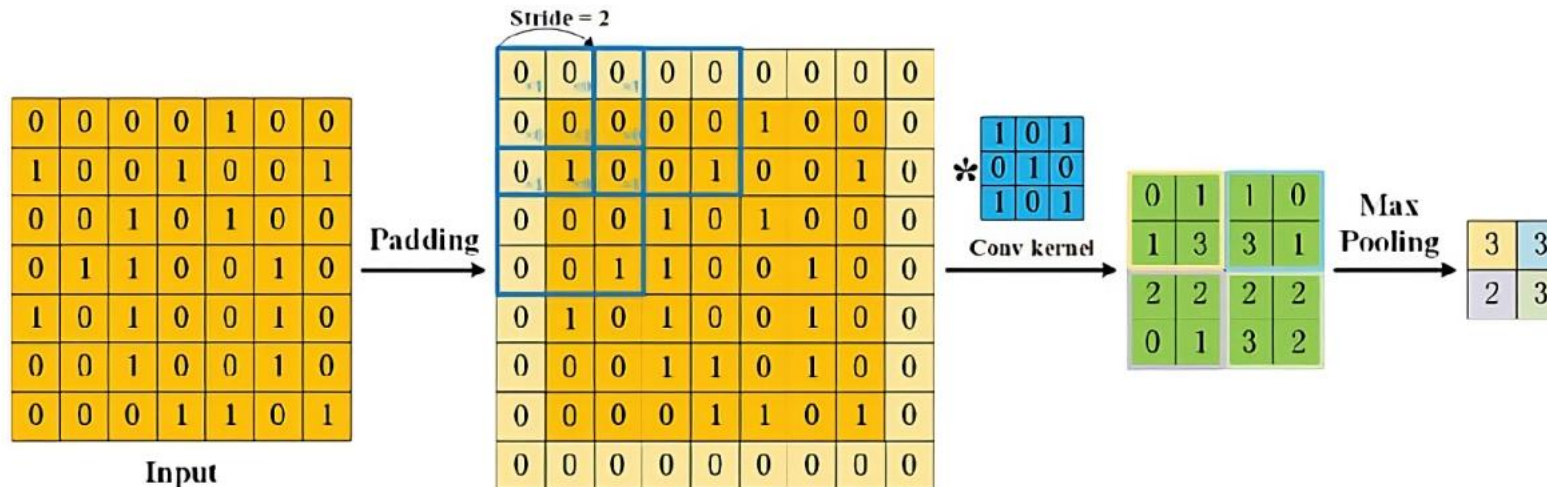
We have two controls for the convolution

Stride

- How many pixels to "step" the filter at a time.
- Default stride=1 (moves 1 pixel at a time).
- stride=2 will skip every other pixel, [downsampling the image](#) (output will be half the size).

Padding

- Convolutions “shrink” the image (a 5x5 input with 3x3 filter gives a 3x3 output).
- We pad the input with zeros around the border to preserve its size
- Rule of thumb: For a $K \times K$ filter, use padding = $(K-1)/2$ to keep the size the same.



Conv2D in PyTorch: nn.Conv2d

```
import torch
import torch.nn as nn
# Input tensor: [batch, channels, height, width]
x = torch.randn(1, 3, 64, 64) # 1 image, 3 channels, 64
    x64 pixels
# Conv2d with stride and padding
conv2d = nn.Conv2d(
    in_channels=3,          # RGB input
    out_channels=32,        # 32 filters
    kernel_size=3,          # 3x3 filters
    stride=2,               # Stride: move 2 pixels at a
        time
    padding=1               # Padding: pad height and width
        dimensions
)
output = conv2d(x)
print(f"Input shape: {x.shape}")      # [1, 3, 64, 64]
print(f"Output shape: {output.shape}") # [1, 32, 32, 32]
```

Calculating Output Size

This is a critical formula for designing CNNs.

Given: W_{in} = Input Width, K = Kernel (Filter) Size, P = Padding, S = Stride

The output width W_{out} is:
$$W_{out} = \frac{W_{in} - K + 2P}{S} + 1$$

The same formula applies to the height H .

Example: Input is 32×32 . Filter is 3×3 ($K = 3$). Stride is 1 ($S = 1$).

- No Padding ($P = 0$): $W_{out} = (32 - 3 + 0)/1 + 1 = 30$. Output is 30×30 (it shrinks).
- With "Same" Padding ($P = 1$): $W_{out} = (32 - 3 + 2)/1 + 1 = 32$. Output is 32×32 (it stays the same)

Pooling Layers

Idea: Aggressively downsample the feature map.

- Max Pooling ([nn.MaxPool2d](#))
 - Takes a window (e.g., 2×2) and outputs the maximum value from that window.
 - **Effect: Selects the strongest feature signal.**

```
import torch.nn as nn
# Max Pooling
max_pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
```

- Average Pooling ([nn.AvgPool2d](#))
 - Takes a 2×2 window and outputs the average value.
 - Effect: Blurs/smooths the features. Less common, but used in ResNet heads.

```
import torch.nn as nn
# Average Pooling
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2, padding=0)
```

Pooling has no learnable parameters.

Why Pooling?

Computational Efficiency:

- A 2×2 pooling layer (with stride 2) cuts the feature map size by 75% (e.g., $32 \times 32 \rightarrow 16 \times 16$).
- This drastically reduces the number of parameters and computations needed in the next layer.

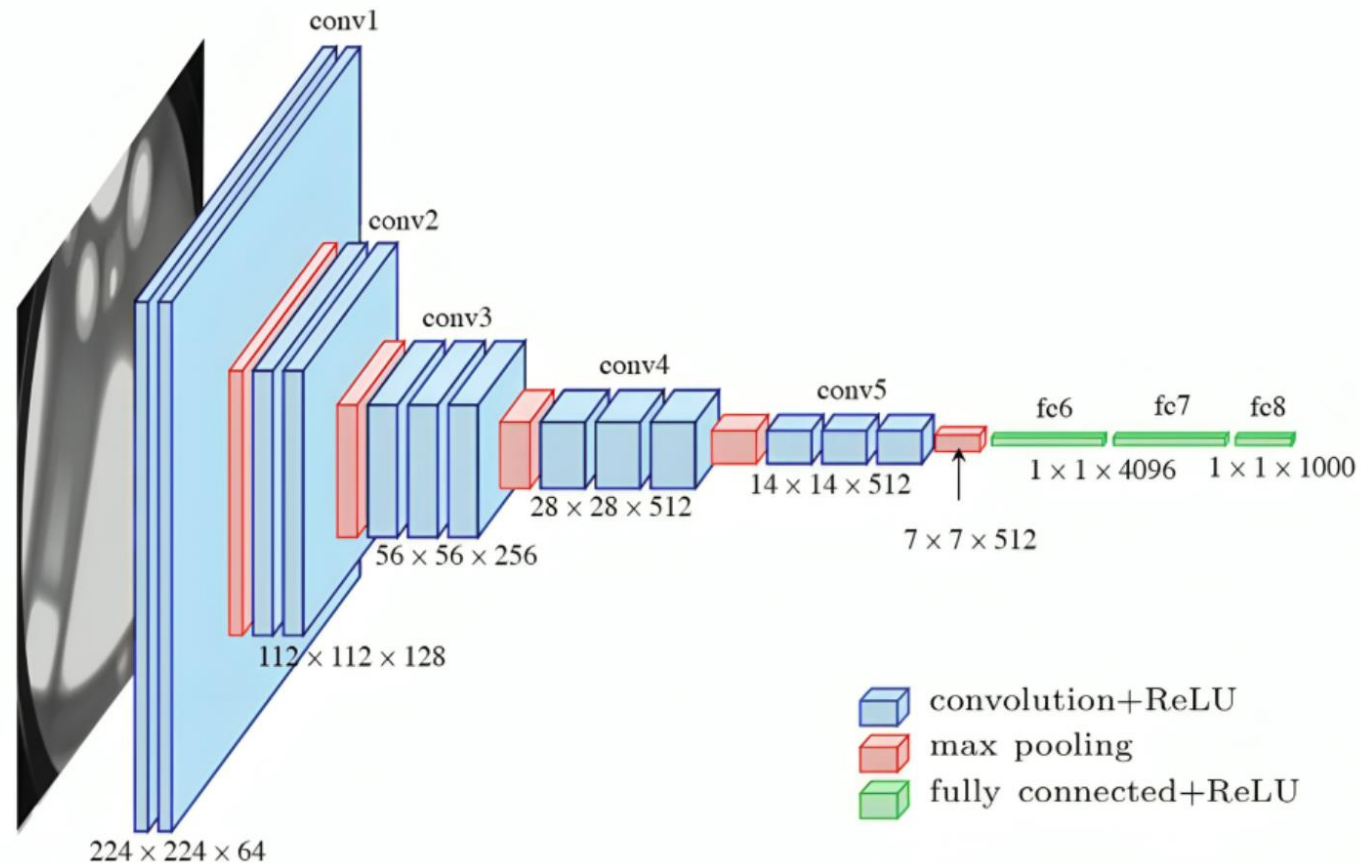
Local Translation Invariance:

- By taking the max over a 2×2 region, the network becomes "less sensitive" to the exact position of the feature.
- An edge in position (4, 4) and an edge in (4, 5) might both result in the same max output.
- This helps the model generalize better

A Full CNN Architecture (e.g., VGG-style)

A typical CNN architecture is a repeating pattern:

(CONV → ReLU → POOL) × N → (FC → ReLU) × M → FC (Output) :



A Simple CNN in PyTorch

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Convolutional Base
        self.conv_base = nn.Sequential(
            # Input: 3 x 32 x 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2), # -> 32 x 16 x 16
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2) # -> 64 x 8 x 8
        )
        # Classifier Head
        self.classifier = nn.Sequential(
            nn.Flatten(), # -> 64 * 8 * 8 = 4096
            nn.Linear(64 * 8 * 8, 512),
            nn.ReLU(),
            nn.Linear(512, num_classes)
        )
    def forward(self, x):
        x = self.conv_base(x)
        x = self.classifier(x)
        return x
```

■ Motivation for Residual Connections

We know stacking layers builds better features. Why not stack 100 layers? Or 1000?

Vanishing Gradients

- With more layers, the gradients for early layers "vanish", and they stop learning.
- ReLU helps, but doesn't fix it entirely

Degradation

- Simply stacking 100 layers often performs worse than a 20-layer version.
- The network has trouble learning a simple identity mapping (i.e., just passing the input through $H(x) = x$)

Residual Connections (ResNet)

- The "skip connection" adds the original input x to the output of the block.
- If the best solution is an identity map ($H(x) = x$), the network can easily learn to set the weights in $F(x)$ to zero
- It also allows gradients to flow directly back through the skip connection, solving the vanishing gradient problem.

Plain Block

$x \rightarrow \text{Conv Block} \rightarrow H(x)$

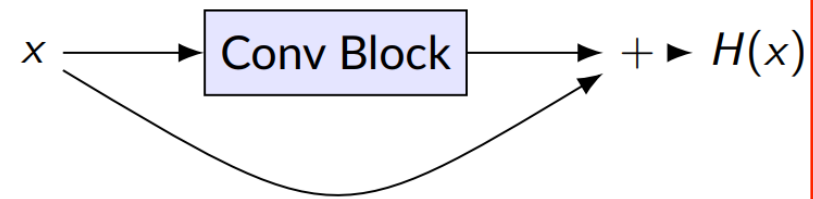
The block must learn the entire transformation $H(x)$.



Residual Block

$$H(x) = F(x) + x$$

The block only learns the *residual* $F(x)$.



ResNet Block in PyTorch

```
class ResidualBlock(nn.Module):  
    def __init__(self, channels):  
        super().__init__()  
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3,  
                                padding=1)  
        self.bn1 = nn.BatchNorm2d(channels)  
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3,  
                                padding=1)  
        self.bn2 = nn.BatchNorm2d(channels)
```

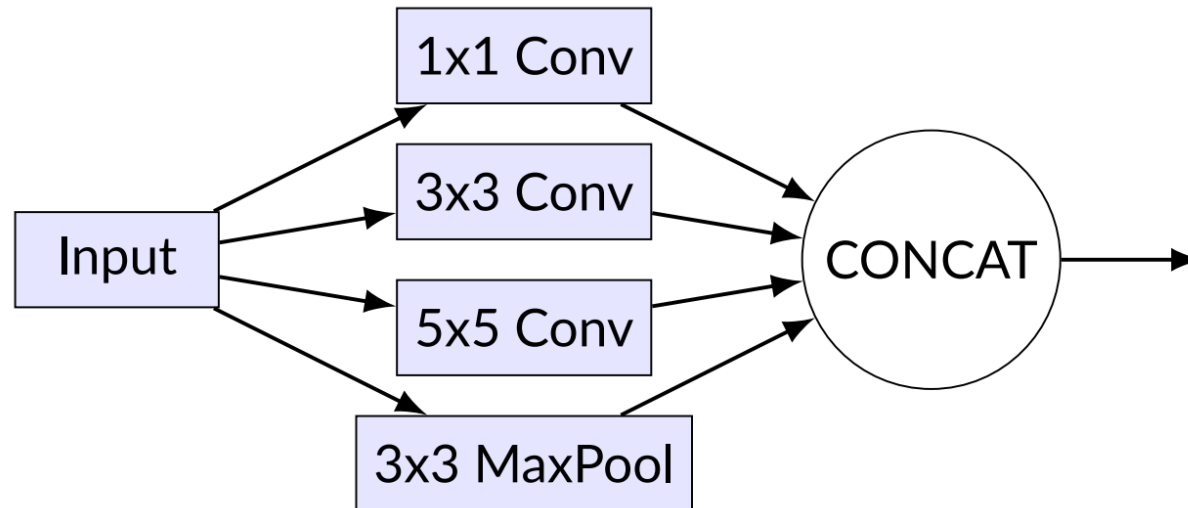
```
    def forward(self, x):  
        # Store the original input (the residual)  
        residual = x  
        # Pass through the block  
        out = F.relu(self.bn1(self.conv1(x)))  
        out = self.bn2(self.conv2(out))  
        # Add the residual back  
        out = out + residual  
        # Apply final activation  
        out = F.relu(out)  
        return out
```

Common Blocks: Inception Module

Problem: What kernel size is best? 3x3? 5x5? 1x1?

Idea (GoogLeNet / Inception): Why not all of them?

- An "Inception module" runs multiple convolutional branches in parallel and concatenates their outputs.
- 1x1 Conv: A cheap way to reduce the number of channels (a "bottleneck") before the expensive 3x3 and 5x5 convs.
- This allows the network to be "wider" instead of just "deeper".



Common Blocks: Depthwise Separable Conv

Problem: A standard 3×3 convolution on a 64-channel input is expensive. It requires 64 different $3 \times 3 \times 64$ filters.

Idea (MobileNet): Split the convolution into two steps

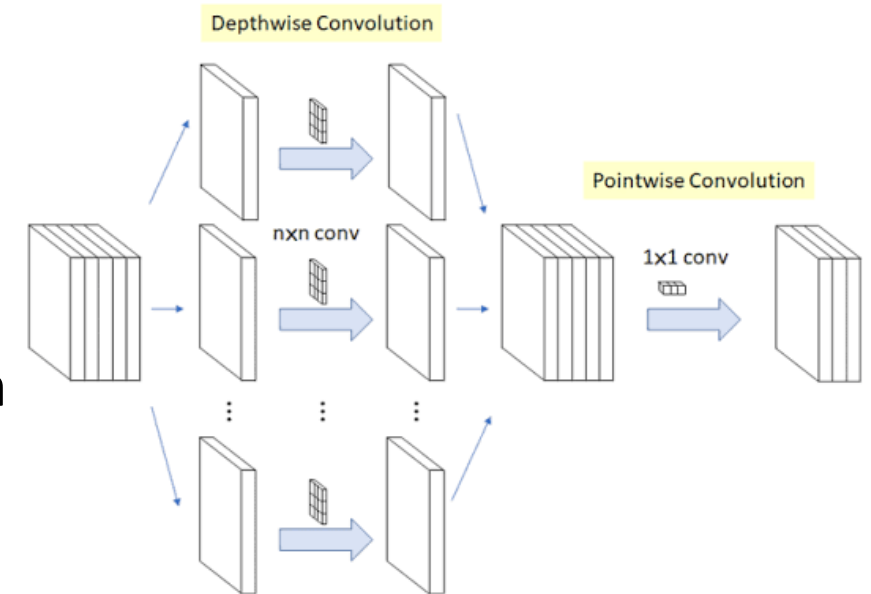
- Depthwise Convolution

- Apply one 3×3 filter to each input channel independently.
- This captures spatial patterns (like “edge”) but doesn’t mix channel information.
- (Input: $C \times H \times W$) $\rightarrow$ (Output: $C \times H \times W$)

- Pointwise Convolution (1x1 Conv)

- Apply a 1×1 convolution to mix the channel information.
- This is just a linear layer applied at every pixel.
- (Input: $C \times H \times W$) $\rightarrow$ (Output: $C_{out} \times H \times W$)

More parameter-efficient than a standard convolution



In-Class Practice (Submit to Canvas)

Task: Build a CNN classifier for CIFAR-10 using modern CNN building blocks.

Step 1: Setup and Data Loading

- Load the CIFAR-10 dataset using `torchvision.datasets.CIFAR10`
- Use the standard train/test split (50k/10k samples)
- Apply normalization transforms

Step 2: Model Architecture

- Create a CNN with at least 2 different types of blocks: **ResNet blocks** and **Inception modules**
- Include **BatchNorm2d** after each convolution layer
- Use **Dropout (p=0.3)** in the classifier head
- Implement **Global Average Pooling** instead of Flatten+Linear for the final layers

Step 3: Training Configuration

- Use **`nn.CrossEntropyLoss`** as your loss function
- Implement **Adam optimizer** with **learning rate 0.001**
- Train for at least 20 epochs with **learning rate scheduling** (reduce by half every 5 epochs), output the train loss and learning rate of every epoch
- Save the trained model ('mymodel.pth')
- Load model & evaluate on test split, compared with *a pre-trained mobilenetv2_x0_5* from the link:
<https://github.com/chenyaofo/pytorch-cifar-models>



Sequence Modeling and RNN

What is Sequence Data?

- Data where order matters.
- Text: "man bites dog" vs. "dog bites man"
- Time Series: Stock prices, weather data
- Audio: Waveforms

Why can't CNNs or MLPs handle sequences?

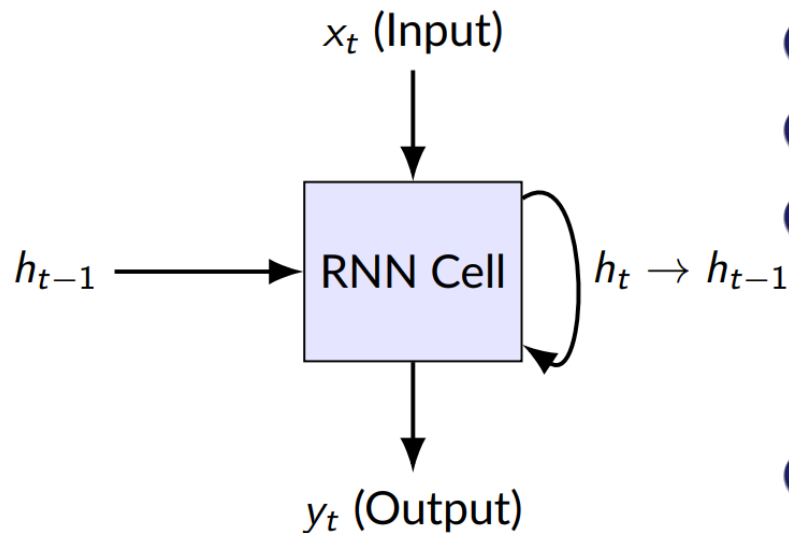
- **No sense of "time" or order:** They treat every input as independent. A shuffled sentence would look the same to an MLP.
- **Fixed-size input:** An MLP trained on 10-word sentences can't handle a 20-word sentence. A CNN has a fixed input size.
- **No "memory":** When processing the 10th word, they have no memory of the first 9 words.

Solution: Recurrent Neural Networks (RNNs)

Idea: A network with a "loop" that maintains a "memory" or "state"

The hidden state h_t is the network's "memory". It's a summary of everything it has seen up to time t .

The RNN Cell



(Folded)

At each time step t , the RNN cell:

- 1 Takes the **current input** x_t .
- 2 Takes the **previous hidden state** h_{t-1} .
- 3 Computes the **new hidden state** h_t :

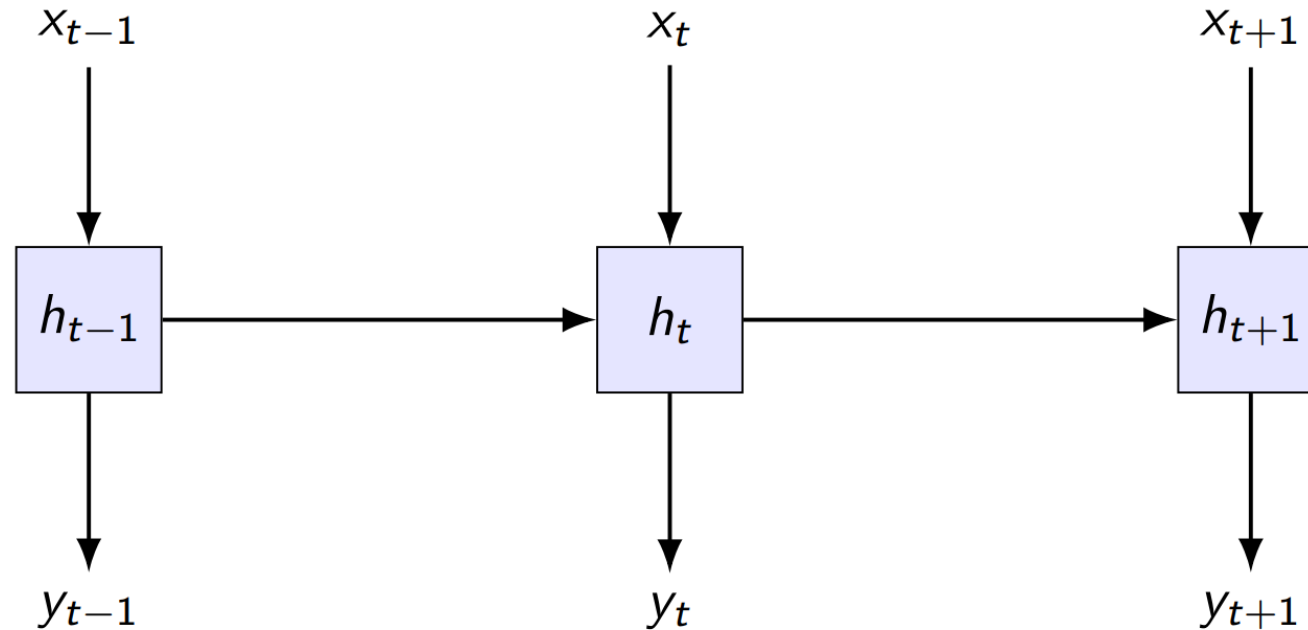
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- 4 (Optionally) computes an output y_t :

$$y_t = W_{hy}h_t + b_y$$

Unrolling the RNN

- Think of the RNN as a deep network "unrolled" through time.
- An RNN is just a very deep network where each layer ($t, t + 1, \dots$) shares the same weights
- Training is done with Backpropagation Through Time (BPTT), which is just standard backpropagation on the unrolled graph.



RNN in PyTorch: nn.RNN

```
# Hyperparameters
input_size = 10    # e.g., dimension of word embedding
hidden_size = 20   # dimension of our "memory"
num_layers = 2     # number of stacked RNN layers
batch_size = 5
seq_len = 7        # length of our sentences
# Initialize the RNN layer
rnn = nn.RNN(input_size=input_size,
              hidden_size=hidden_size,
              num_layers=num_layers,
              batch_first=True) # Makes shape [Batch, Seq, Feat]
# Create dummy input data
# Shape: [batch_size, sequence_length, input_size]
dummy_input = torch.rand(batch_size, seq_len, input_size)
# Pass it through the RNN
# output: hidden state for *every* time step
# h_n: hidden state for the *last* time step
output, h_n = rnn(dummy_input)
print("Input shape:", dummy_input.shape)
print("Output shape:", output.shape)
print("Last hidden shape:", h_n.shape)
```

Problem: The RNN's "memory" h_t is bad. It has to be used for two jobs at once:

- Be the output for the current time step (for prediction).
- Be the "memory" for all future time steps.

Solution: Gated RNNs

- create a separate, protected "memory" (the Cell State, c_t) that is separate from the output (h_t).
- Use "gates" (multiplicative σ functions) to control the flow of information into and out of this protected memory

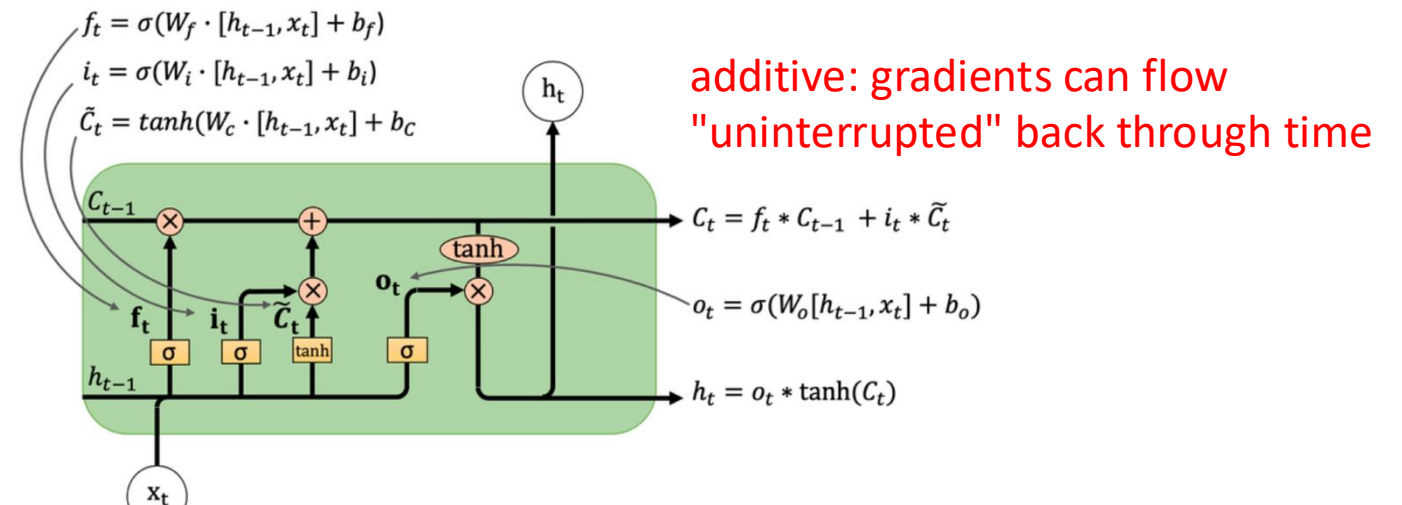
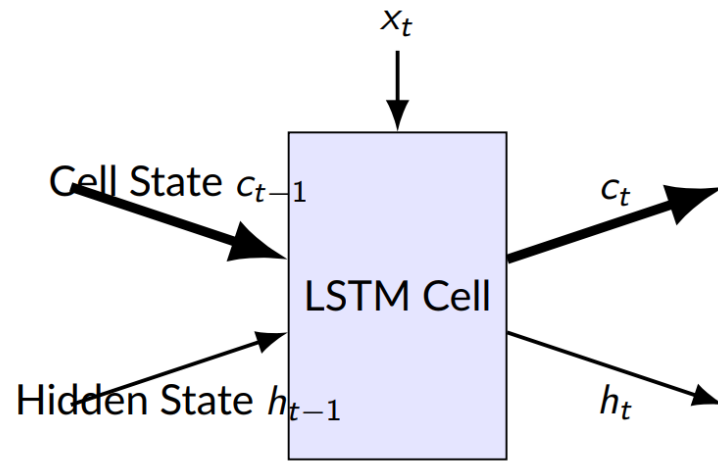
This leads to:

- **GRU** (Gated Recurrent Unit)
- **LSTM** (Long Short-Term Memory)

Long Short-Term Memory (LSTM)

An LSTM cell is a more complex RNN cell with two states:

- h_t (Hidden State) The "output" state. Used for predictions at time t .
- c_t (Cell State) The "memory" state. A "conveyor belt" for information.
- The network uses "gates" to read from, erase from, and write to cell state.
- Forget Gate (f_t): How much of the old memory c_{t-1} should we forget?
- Input Gate (i_t): How much of the new information should we add?
- Output Gate (o_t): "What part of the (updated) cell state c_t should we output as the new hidden state h_t ?"



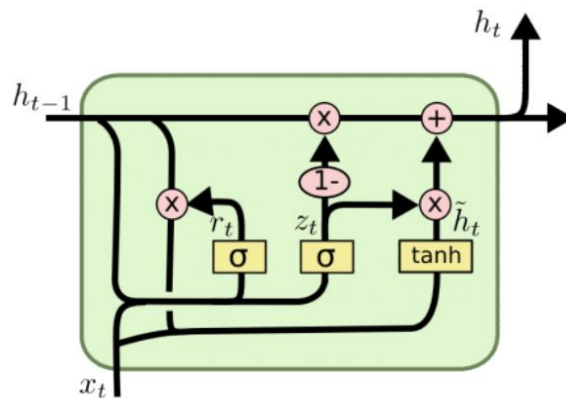
```
# Initialize an LSTM layer
# API is identical to nn.RNN
lstm = nn.LSTM(input_size=input_size,
               hidden_size=hidden_size,
               num_layers=num_layers,
               batch_first=True) # Very important!
```

```
# LSTM returns (output, (h_n, c_n))
# h_n = final hidden state, c_n = final cell state
output, (h_n, c_n) = lstm(dummy_input)
```

Gated Recurrent Unit (GRU)

GRU works just as well as LSTM on many tasks, but is faster to train (**fewer parameters**).

- It merges the hidden state and cell state into **one hidden state h_t** .
- It only has **two gates**:
 - Update Gate (z_t): Combines the "forget" and "input" gates. Decides how much to keep from h_{t-1} vs. how much to add from the new candidate $\tilde{h}_t$.
 - Reset Gate (r_t): Decides how much of the previous memory h_{t-1} to use when calculating the new candidate $\tilde{h}_t$.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

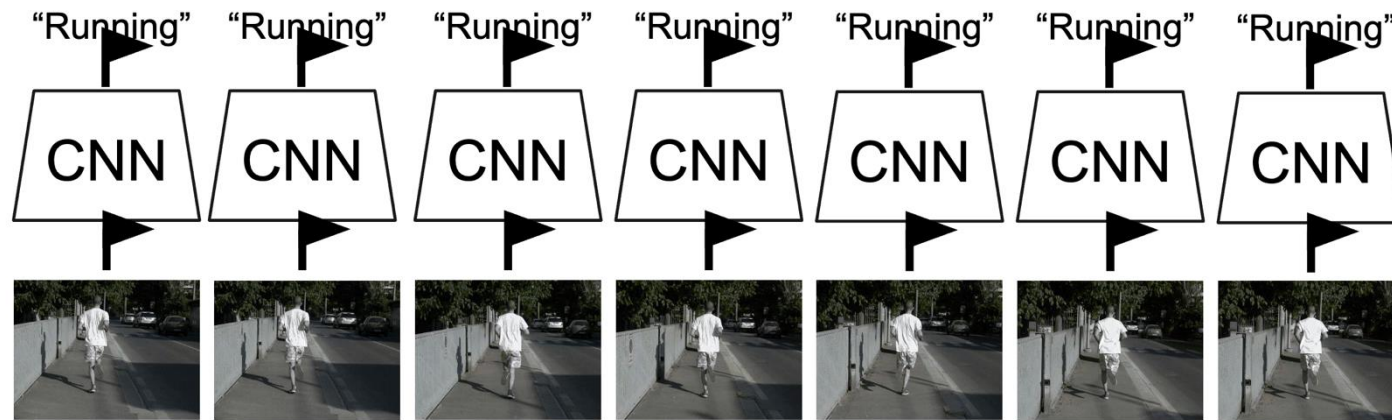
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

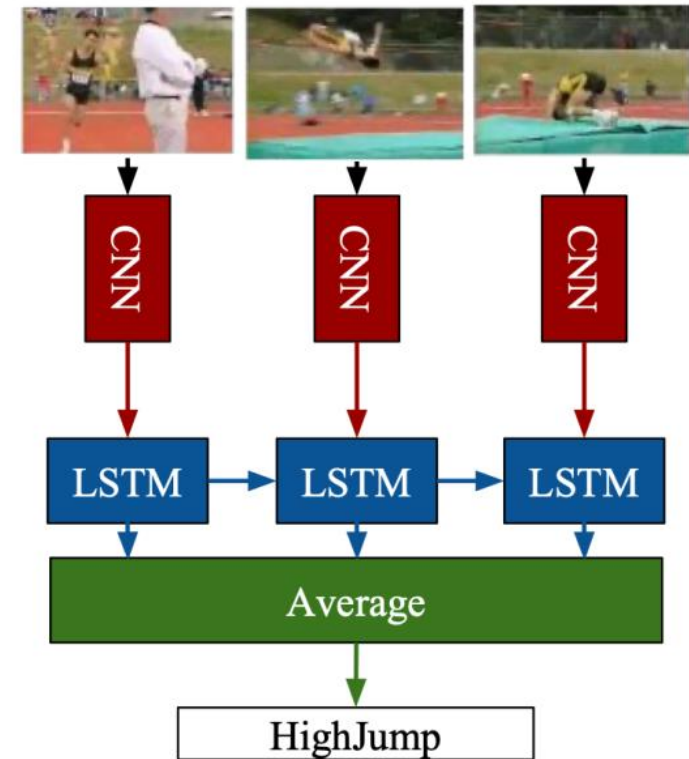
```
# Initialize a GRU layer
gru = nn.GRU(input_size=input_size,
             hidden_size=hidden_size,
             num_layers=num_layers,
             batch_first=True)
```

```
# GRU returns (output, h_n)
output, h_n = gru(dummy_input)
```

Example: Video Understanding



Activity Recognition Sequences in the Input



[CVPR 2014] Large-scale Video Classification with Convolutional Neural Network

[CVPR 2015] Long-term Recurrent Convolutional Networks for Visual Recognition and Description

CNN+RNN in PyTorch

```
import torch
import torch.nn as nn

class CNNLSTM(nn.Module):
    def __init__(self, num_classes=10, feature_dim=128,
                  hidden_dim=256, num_layers=1):
        super().__init__()

        # ----- CNN: extract features from each frame -----
        self.cnn = nn.Sequential(
            # Input frame: 3 x 32 x 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2), # -> 32 x 16 x 16

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2), # -> 64 x 8 x 8

            nn.Flatten(), # -> 64 * 8 * 8
            nn.Linear(64 * 8 * 8, feature_dim),
            nn.ReLU()
        )

        # ----- LSTM: model temporal dependency across frames -----
        self.lstm = nn.LSTM(
            input_size=feature_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True
        )

        # ----- Classifier: map video feature to class scores -----
        self.classifier = nn.Linear(hidden_dim, num_classes)
```

```
def forward(self, x):
    """
    x: [batch, time, channels, height, width]
       e.g., [B, T, 3, 32, 32]
    """

    B, T, C, H, W = x.shape

    # 1. Merge batch and time dimensions
    # Treat all frames as independent images for CNN
    x = x.view(B * T, C, H, W)
    # x: [B*T, 3, 32, 32]

    # 2. Extract frame-level CNN features
    frame_features = self.cnn(x)
    # frame_features: [B*T, feature_dim]

    # 3. Reshape back to video sequence
    frame_features = frame_features.view(B, T, -1)
    # frame_features: [B, T, feature_dim]

    # 4. Feed frame feature sequence into LSTM
    lstm_outputs, _ = self.lstm(frame_features)
    # lstm_outputs: [B, T, hidden_dim]

    # 5. Average over time
    video_feature = lstm_outputs.mean(dim=1)
    # video_feature: [B, hidden_dim]

    # 6. Predict action class
    logits = self.classifier(video_feature)
    # logits: [B, num_classes]

    return logits
```



Language Modeling and Transformer

What is Language Modeling?

- **Language Modeling:** The task of capturing the statistical properties and patterns of human language.
- At its core, it's about answering the question: "What comes next?"

Example

Given the context: "The students opened their..."

A system must assign probabilities:

- books (high probability)
- laptops (medium probability)
- bananas (very low probability)

What is Language Model?

- Language Model (LM) is the system, model, or algorithm that performs the Language Modeling Task.
- To do this, the system must learn a probability distribution over all possible sequences.

What is Language Model?

- **Tokens:** The basic building blocks (symbols, words, or subwords) of LMs
- **Language Model (P):** A probability distribution over all possible sequences of tokens
- **Core Task:** The model must compute the conditional probability of the **next** token given the preceding history.

$$P(y_t|y^{t-1})$$

- **Sequence Probability:** The probability of an entire sentence is the product of these conditional probabilities

$$P(y^T) = \prod_{t=1}^T P(y_t|y^{t-1})$$

How is an LM Learned? (MLE)

Hypothesis Class (H): We first choose an architecture (a set of models h_θ)

Training Data (y^T): A large corpus of text.

Learning Objective: Maximum Likelihood Estimation (MLE)

- Find the parameters that maximize the probability of observing the training data

$$\hat{\theta} := \arg \max_{\theta \in \Theta} \prod_{t=1}^T \underbrace{h_\theta(y^{t-1})[y_t]}_{\text{represent the model's predicted conditional distribution}}$$

represent the model's predicted conditional distribution

- In Practice (Log Loss):** This is equivalent to minimizing the Negative Log-Likelihood (Logarithmic Loss)

$$\hat{\theta} = \arg \min_{\theta \in \Theta} \sum_{t=1}^T -\log h_\theta(y^{t-1})[y_t]$$

Setup

```
import torch
import torch.nn as nn
import torch.optim as optim
# 1. Hypothesis Class (e.g., a Transformer)
# model = MyTransformerModel(vocab_size, ...)
# h_theta
model = ... # (Instantiate our model)

# 2. Training Data
# context = "The students opened" -> [5, 27, 89]
# target = "their" -> [101]
context_ids = torch.tensor([[5, 27, 89]]) # y^(t-1)
target_ids = torch.tensor([101]) # y_t

# 3. Learning Objective (Negative Log-Likelihood)
# nn.CrossEntropyLoss is the key!
# It directly implements: -log h_theta(y^(t-1))[y_t]
loss_function = nn.CrossEntropyLoss()
# 4. Optimization Algorithm (e.g., Adam)
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

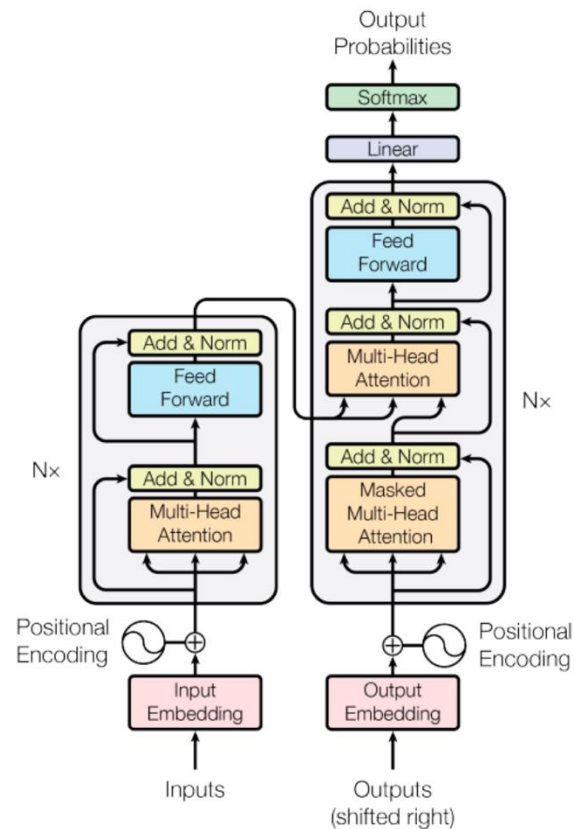
Training Step

```
# --- A Single Training Step ---  
# Zero out gradients from last step  
optimizer.zero_grad()  
  
# Forward pass: Get predictions (logits)  
# Logits =  $h_{\theta}(y^{(t-1)})$   
logits = model(context_ids)  
# E.g., logits shape: [1, vocab_size]  
  
# Compute the Log Loss  
# This is the core of MLE  
loss = loss_function(logits, target_ids)  
  
# Backward pass: Compute gradients  
loss.backward()  
# Update model parameters ( $\theta$ )  
optimizer.step()
```

SOTA LM: Transformer Models

The key to learning is choosing a good architecture for the hypothesis class H

Transformer Models: Use self-attention to process sequences in parallel.



Component 1: Input Embedding

This is the first **learnable layer** of the network, converting discrete tokens into continuous vectors.

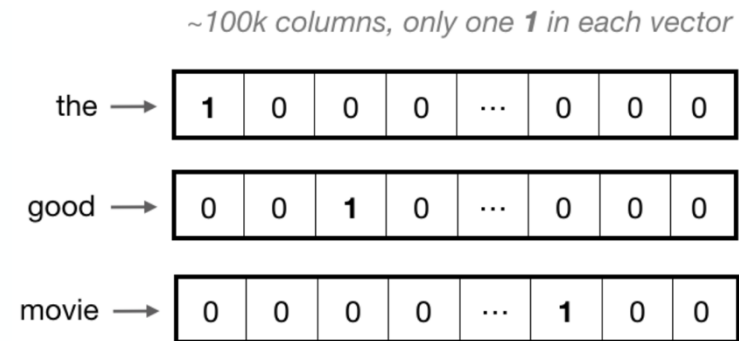
Vocabulary (Y): The set of all possible tokens.

- e.g., In GPT-3, $|Y| \sim 50,000$

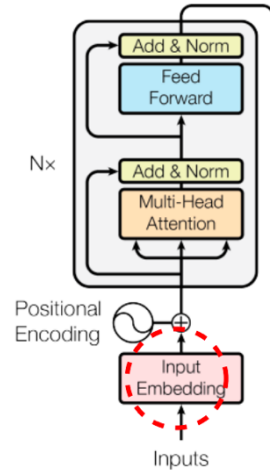
Large generative models (e.g., large language models, diffusion models) have shown remarkable performance, but they require a massive amount of computational resources. To make them more accessible, it is crucial to improve their efficiency. This course will introduce efficient AI computing techniques that enable powerful deep learning applications on resource-constrained devices. Topics include model compression, pruning, quantization, neural architecture search, distributed training, data/model parallelism, gradient compression, and on-device fine-tuning. It also introduces application-specific acceleration techniques for large language models, diffusion models, video recognition, and point cloud. This course will also cover topics about quantum machine learning. Students will get hands-on experience deploying large language models (e.g., LLaMA 2) on a laptop.

TEXT TOKEN IDS

A tokenizer maps a word to one/multiple tokens



One-hot Encoding: Representing each word as a vector with as many values as there are words in the vocabulary. Each column in a vector represents one possible word in a vocabulary.



Component 1: Input Embedding

This is the first **learnable layer** of the network, converting discrete tokens into continuous vectors.

Vocabulary (Y): The set of all possible tokens.

- e.g., In GPT-3, $|Y| \sim 50,000$

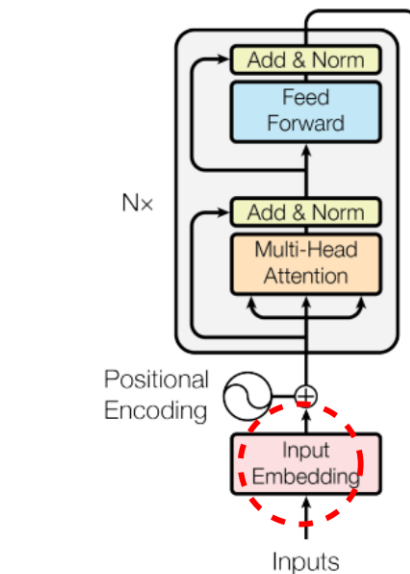
Embedding Matrix (M): A large, **learnable** matrix $M \in \mathbb{R}^{d \times |Y|}$.

- d is the embedding dimension (e.g., $d = 12,288$ for GPT-3).

Mapping:

- Each token y (represented as a one-hot vector e_y) is mapped to a dense vector x_y .

$$x_y := Me_y \in \mathbb{R}^d$$



the	→	0.2	0.4	-0.1
good	→	0.7	-0.5	0.3
movie	→	0.1	0.2	0.6

Token Embedding in PyTorch: nn.Embedding

```
import torch
import torch.nn as nn
import math
# --- Model Hyperparameters ---
vocab_size = 50000 # |Y|
d_model = 768 # d (embedding dimension)
max_seq_len = 1024 # Maximum sequence length
# --- Token Embedding Layer ---
# This is our learnable matrix M
# It's a simple lookup table: index -> vector
token_embedding_layer = nn.Embedding(vocab_size, d_model)
# --- Example Usage ---
# Batch of 2 sentences, 5 tokens each
# (e.g., [ "The", "cat", "sat", "on", "the" ],
#        [ "A", "dog", "ran", ".", "<pad>" ])
input_ids = torch.randint(0, vocab_size, (2, 5))
# input_ids.shape: [batch_size=2, seq_len=5]
# Pass token IDs through the embedding layer
#  $X = M * e_y$ 
token_embeddings = token_embedding_layer(input_ids)
# token_embeddings.shape: [2, 5, d_model=768]
```

■ Component 2: Positional Encoding

Other component in Transformer cannot capture word order

Solution: Positional Embeddings

- A vector $p_i \in \mathbb{R}^d$ is defined for each position i .
- This P matrix is **added** to the input token embeddings X

$$X' = X + P \in \mathbb{R}^{t \times d}, t = \text{seq_len}$$

Positional Encoding in PyTorch

```
import torch
import torch.nn as nn

# --- Model Hyperparameters ---
vocab_size = 50000      # vocabulary size
d_model = 768           # embedding dimension
max_seq_len = 1024      # maximum sequence length

# --- Token Embedding Layer ---
# Converts token IDs into token vectors
token_embedding_layer = nn.Embedding(vocab_size, d_model)

# --- Positional Embedding Layer ---
# Converts position IDs into position vectors
position_embedding_layer = nn.Embedding(max_seq_len, d_model)

# --- Example Usage ---
# Batch of 2 sentences, 5 tokens each
# e.g., [ ["The", "cat", "sat", "on", "the"],
#         ["A", "dog", "ran", ".", "<pad>"] ]
input_ids = torch.randint(0, vocab_size, (2, 5))
# input_ids.shape: [batch_size=2, seq_len=5]

batch_size, seq_len = input_ids.shape

# Create position IDs: [0, 1, 2, 3, 4]
position_ids = torch.arange(seq_len)
# position_ids.shape: [seq_len]

# Expand to match the batch dimension
position_ids = position_ids.unsqueeze(0).expand(batch_size, seq_len)
# position_ids.shape: [batch_size=2, seq_len=5]

# Get token embeddings
token_embeddings = token_embedding_layer(input_ids)
# token_embeddings.shape: [2, 5, d_model=768]

# Get positional embeddings
position_embeddings = position_embedding_layer(position_ids)
# position_embeddings.shape: [2, 5, d_model=768]

# Add them together
input_embeddings = token_embeddings + position_embeddings
# input_embeddings.shape: [2, 5, d_model=768]
```

Task: Use PyTorch to compare the representations of:

- man eats dog
- dog eats man

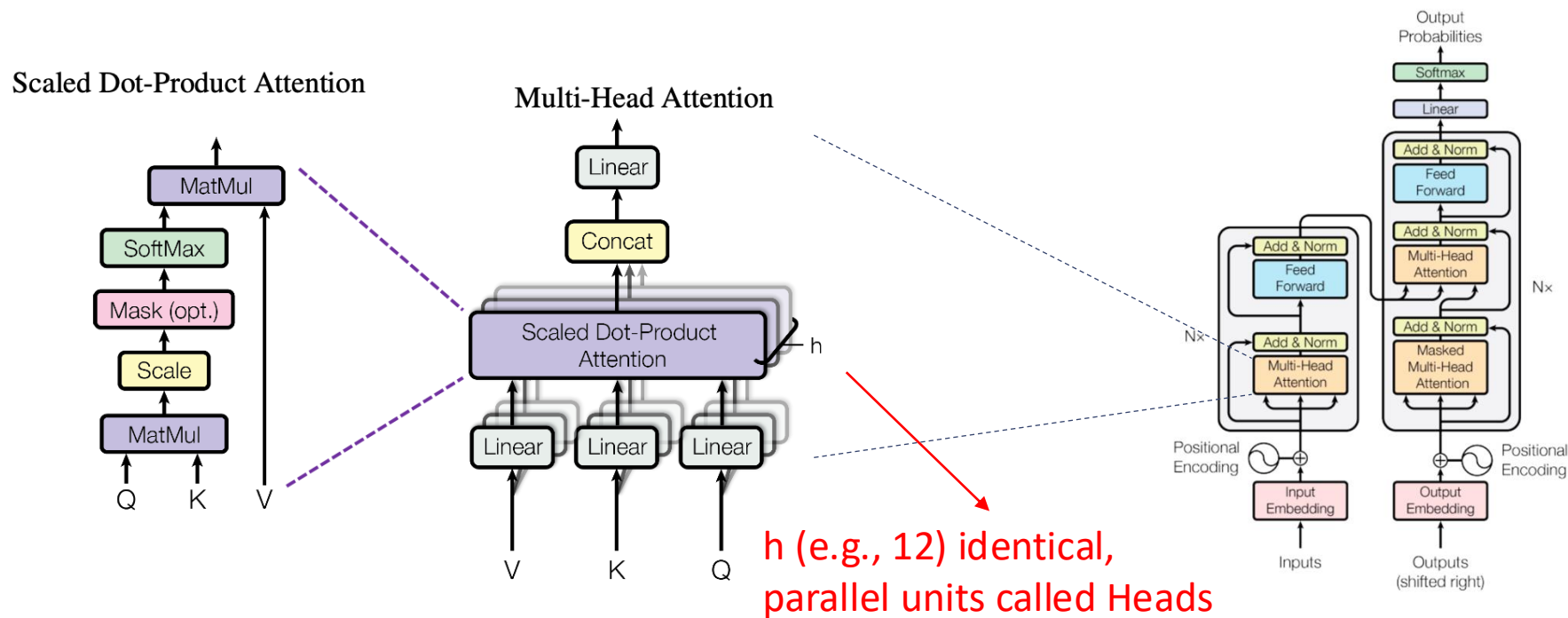
Steps:

- Create a tiny vocabulary: man, eats, dog.
- Convert both sentences into token IDs.
- Use `nn.Embedding` to obtain token embeddings.
- Average token embeddings to get sentence representations, and compare them.
- Add positional embeddings and compare the sentence representations again.

Component 3: Multi-Head Attention (MHA)

MHA is the core attention mechanism of the Transformer

- **Parallel:** It's highly parallelizable, making it efficient for modern hardware (GPUs)
- To understand the full MHA, we must first understand its core building block: the **Scaled Dot-Product Attention** (i.e., a “single head”)



Scaled Dot-Product Attention (“Head”)

The core logic of a single attention head i

Project: Create Query, Key, and Value matrices from the input $X \in \mathbb{R}^{t \times d}$ ($t=\text{seq_len}$, $d=\text{model_dim}$) (after position embedding) using learnable weights $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d'}$ ($d'=\text{head_dim} = d/h$).

- $Q_i := XW_q^i \in [t, d']$
- $K_i := XW_k^i \in [t, d']$
- $V_i := XW_v^i \in [t, d']$

Calculate Scores: Compute scaled dot-product attention scores. “How much does each token ‘query’ match each ‘key’?”

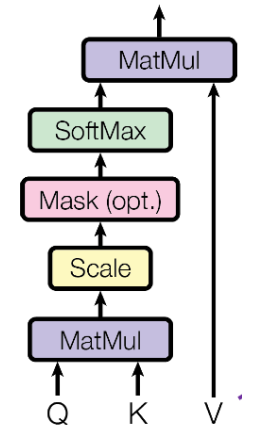
$$S_i := \frac{(Q_i K_i^\top)}{\sqrt{d'}} \in \mathbb{R}^{t \times t}$$

Get Weights (Softmax): Apply softmax row-wise to get attention weights (probabilities).

$$P_i := \text{softmax}(S_i) \in \mathbb{R}^{t \times t}$$

Get Output: Weighted sum of Values based on attention weights.

$$X_i := P_i V_i \in \mathbb{R}^{t \times d'}$$



Why Scale the Dot Products?

Problem:

- The dot product can grow very large in magnitude, especially if the dimension d' is large.
- Large inputs (e.g., $[10, 80, 5, -100]$) pushed into a 'softmax' function cause it to "saturate".
 - 'softmax($[1, 2]$)' $\rightarrow$ $[0.27, 0.73]$ (Soft probabilities)
 - 'softmax($[10, 80]$)' $\rightarrow$ $[0.0, 1.0]$ (Hard, one-hot probabilities)
- When 'softmax' saturates to 0 or 1, its gradient becomes extremely small, Vanishing gradients make training very slow or impossible.

Solution:

- Assume q and k have mean 0 and variance 1, thus, the dot product has mean 0 but variance d' .
- To counteract this, we divide by the standard deviation $\sqrt{d'}$ to rescales the variance back to 1, keeping the inputs to 'softmax' small and mitigating saturation.

Scaled Dot-Product Attention in PyTorch

```
import torch.nn.functional as F
import math
def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q, K, V shape: [Batch, Num_Heads, Seq_Len, Head_Dim]
    # 1. Get head dimension
    d_k = Q.size(-1)
    # 2. Calculate Scores: (B, H, T, T)
    #      (B, H, T, D) @ (B, H, D, T) -> (B, H, T, T)
    scores = torch.matmul(Q, K.transpose(-2, -1))
    # 3. Scale: (The focus of the last slide)
    scores = scores / math.sqrt(d_k)
    # 4. (Optional) Apply Mask
    #      (e.g., for padding or for decoder look-ahead)
    if mask is not None:
        # Fills -infinity where mask is 1
        scores = scores.masked_fill(mask == 1, -1e9)
    # 5. Get Weights (Softmax)
    weights = F.softmax(scores, dim=-1)
    # 6. Get Output (Weighted sum of V)
    #      (B, H, T, T) @ (B, H, T, D) -> (B, H, T, D)
    output = torch.matmul(weights, V)
    return output, weights
```

In-Class Practice

Problem:

- We process sentences in batches. Sentences have different lengths.
- We pad short sentences with a 'pad_token' (e.g., ID 0) so all tensors in the batch are the same size.
- We must not let the model "attend" to these padding tokens..

Task:

- Our 'scaled_dot_product_attention' function fills where 'mask == 1'.
- This means we need a mask that is False (or 0) for real tokens and True (or 1) for pad tokens.
- It must also have the broadcastable shape: [B, 1, 1, T].

```
# Batch of 2 sentences, Seq length
# is 4, 0 is the <pad> token
input_ids = torch.tensor([
    [5, 8, 12, 0],  # "The cat sat"
    [7, 2, 0, 0]   # "A dog"
])
# Shape: [B=2, T=4]
```

Component 3: Multi-Head Attention (MHA)

Linearly project $\mathbf{X}$ to get h different sets of $(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)$.

Run h independent SDPA heads in parallel.

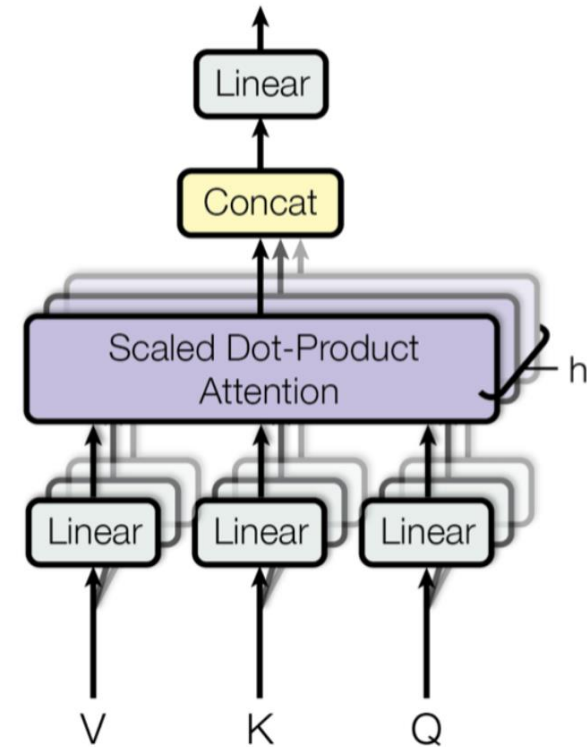
This yields h separate output matrices

$(\mathbf{X}_1, \dots, \mathbf{X}_h)$.

Concatenate all h outputs:

$\text{Concat}(\mathbf{X}_1, \dots, \mathbf{X}_h)$.

Apply a final linear projection $\mathbf{W}^O$.



Multi-Head Attention (MHA) in PyTorch

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, h):
        super().__init__()
        self.d_model = d_model # e.g., 768
        self.h = h # e.g., 12
        self.d_k = d_model // h # e.g., 64

        # --- Learnable Weights ---

        # Linear layers for Q, K, V (W_q, W_k, W_v)
        # We define one big linear layer for each
        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)

        # Final output projection (W_o)
        self.out_linear = nn.Linear(d_model, d_model)

        # (forward method on next slide...)
```

Multi-Head Attention (MHA) in PyTorch

```
def forward(self, x, mask=None):
    # x shape: [Batch, Seq_Len (T), d_model]
    batch_size = x.size(0)
    # 1. Project and split into h heads
    # Q/K/V shape: [B, T, d_model] -> [B, h, T, d_k]
    Q = self.q_linear(x).view(batch_size, -1, self.h, self.d_k).
        transpose(1,2)
    K = self.k_linear(x).view(batch_size, -1, self.h, self.d_k).
        transpose(1,2)
    V = self.v_linear(x).view(batch_size, -1, self.h, self.d_k).
        transpose(1,2)

    # 2. Run all heads in parallel (using our function)
    # new_x shape: [B, h, T, d_k]
    new_x, weights = scaled_dot_product_attention(Q, K, V, mask)

    # 3. Concat and 4. Project
    # [B, h, T, d_k] -> [B, T, h, d_k] -> [B, T, d_model]
    new_x = new_x.transpose(1, 2).contiguous().view(batch_size,
        -1, self.d_model)
    return self.out_linear(new_x)
```

- `view()` does not create new features.
- It only reorganizes the tensor shape.
- Here, it splits the 768-dimensional vector into 12 heads, each with 64 dimensions.

Component 4: Feed-Forward Layer (FFN)

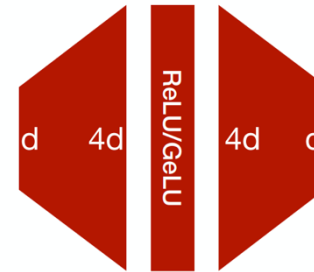
This block is applied after the MHA layer.

- It is a simple two-layer fully connected neural network.
- First layer **expands dimension**: $X \rightarrow X'$ (using W_1, b_1) with ReLU activation.

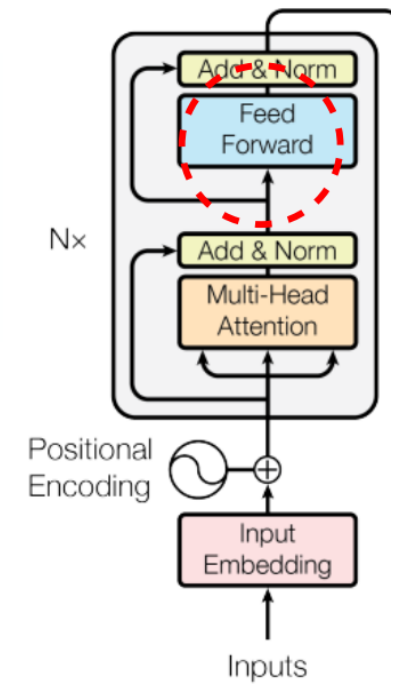
$$X' = \text{ReLU}(XW_1 + b_1)$$

- Second layer **compresses back**: $X' \rightarrow X''$ (using W_2, b_2)

$$X'' = X'W_2 + b_2$$



- e.g., GPT-3 Small: $d = 768 \rightarrow 3072 \rightarrow 768$
- The matrices W_1, W_2 and vectors b_1, b_2 are trainable parameters.



Feed-Forward Layer (FFN) in PyTorch

```
import torch
import torch.nn as nn
class FeedForward(nn.Module):
    # Implements FFN(X) = ReLU(XW1
    # + b1)W2 + b2
    def __init__(self, d_model,
                  d_ff):
        super().__init__()
        # W1 and b1 (e.g., 768 ->
        # 3072)
        self.linear_1 = nn.Linear(
            d_model, d_ff
        )
        self.relu = nn.ReLU()
        # W2 and b2 (e.g., 3072 ->
        # 768)
        self.linear_2 = nn.Linear(
            d_ff, d_model
        )
```

```
    def forward(self, x):
        # x shape: [Batch, Seq_Len
        #             , d_model]
        # (1) x -> [B, T, d_ff]
        x = self.relu(self.
            linear_1(x))
        # (2) x -> [B, T, d_model]
        x = self.linear_2(x)
        return x
# --- Example Usage ---
d_model = 768
d_ff = 3072 # 768 * 4
ffn_block = FeedForward(d_model,
                          d_ff)
# Create a dummy input from MHA
dummy_x = torch.randn(
    2, 10, d_model
) # [B, T, d_model]
output = ffn_block(dummy_x)
# output.shape: [2, 10, 768]
```

■ The Glue: Add & Norm

Each sub-layer (MHA, FFN) is wrapped by two operations:

Residual Connection (Add): Inspired from ResNet

- The input to a layer is **added** to its output: $X + \text{SubLayer}(X)$.
- This “shortcut” allows gradients to flow directly, preventing the vanishing gradient problem and enabling very deep networks.

Layer Normalization (Norm):

- Stabilizes training by normalizing each token's vector $\mathbf{x}_j$ (mean=0, var=1). The mean and variance are computed across the d dimensions, then applies learnable γ (scale) and β (shift) parameters

$$\mu_j = \frac{1}{d} \sum_{k=1}^d \mathbf{x}_j[k], \sigma_j^2 = \frac{1}{d} \sum_{k=1}^d (\mathbf{x}_j[k] - \mu_j)^2 \quad \mathbf{x}'_j = \gamma \odot \left(\frac{\mathbf{x}_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}} \right) + \beta$$

- Normally, applied **before** the residual connection.

Add & Pre-Norm in PyTorch

```
import torch.nn as nn
class ResidualConnection(nn.Module):
    # Implements Pre-Norm:  $X_{final} = X + \text{SubLayer}(\text{LN}(X))$ 
    def __init__(self, sublayer,
                  d_model, dropout):
        super().__init__()
        self.sublayer = sublayer
        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

# --- Example (in your
# --- TransformerLayer __init__)
# self.mha = MultiHeadAttention(...)
# self.ffn = FeedForward(...)

# self.mha_res = ResidualConnection(
#     self.mha, d_model, dropout
# )
# self.ffn_res = ResidualConnection(
#     self.ffn, d_model, dropout
# )
```

Add & Pre-Norm in PyTorch

```
def forward(self, x):  
    # x shape: [B, T, d_model]  
  
    # 1. Normalize first (Pre-Norm)  
    x_norm = self.norm(x)  
  
    # 2. Pass through sublayer (MHA/FFN)  
    sub_output = self.sublayer(x_norm)  
  
    # 3. Apply dropout  
    sub_output = self.dropout(sub_output)  
  
    # 4. Add residual connection  
    return x + sub_output
```

```
# --- Example (in your  
# --- TransformerLayer forward)  
# x = ...  
# x = self.mha_res(x)  
# x = self.ffn_res(x)  
# return x
```

Putting It All Together: A Transformer “Block”

- We have now seen all the “LEGO bricks” (components) needed to build a Transformer layer.
- A single “Transformer Block” (or “Layer”) is just **a stack of these components**.

Input: $\mathbf{X}_{in}$ (from previous layer)

Sub-layer 1 (MHA):

- $\mathbf{X}_{norm1} = \text{LayerNorm}(\mathbf{X}_{in})$
- $\mathbf{X}_{mha} = \text{MultiHeadAttention}(\mathbf{X}_{norm1})$
- $\mathbf{X}_{mid} = \mathbf{X}_{in} + \mathbf{X}_{mha}$ (1st Residual Connection)

Sub-layer 2 (FFN):

- $\mathbf{X}_{norm2} = \text{LayerNorm}(\mathbf{X}_{mid})$
- $\mathbf{X}_{ffn} = \text{FeedForward}(\mathbf{X}_{norm2})$
- $\mathbf{X}_{out} = \mathbf{X}_{mid} + \mathbf{X}_{ffn}$ (2nd Residual Connection)

Output: $\mathbf{X}_{out}$ (to next layer)

- **This entire block is then stacked N times** (e.g., $N = 12$) to build the model

Transformer “Block” in PyTorch

```
class TransformerBlock(nn.Module):
    """
    Implements one full Transformer Block (MHA -> Add&Norm -> FFN ->
    Add&Norm) using the Pre-Norm style.
    """
    def __init__(self, d_model, h, d_ff, dropout_p):
        super().__init__()
        # 1. Multi-Head Attention sub-layer
        self.mha = MultiHeadAttention(d_model, h)
        # 2. Feed-Forward sub-layer
        self.ffn = FeedForward(d_model, d_ff)
        # 3. Two ResidualConnection wrappers
        self.res_conn_1 = ResidualConnection(
            sublayer_module = self.mha,
            d_model = d_model,
            dropout_p = dropout_p
        )
        self.res_conn_2 = ResidualConnection(
            sublayer_module = self.ffn,
            d_model = d_model,
            dropout_p = dropout_p
        )
```

Transformer “Block” in PyTorch

```
def forward(self, x, mask=None):
    # x shape: [B, T, d_model]
    # 1. Apply MHA block:
    # X_mid = X_in + MHA(LN(X_in))
    # We must pass the mask to MHA
    x = self.res_conn_1(x, mask=mask)
    # 2. Apply FFN block:
    # X_out = X_mid + FFN(LN(X_mid))
    # No mask needed for FFN
    x = self.res_conn_2(x)
    return x

# --- Full Model (Conceptual) ---
# self.embedding = ...
# self.rope = ...
# self.transformer_blocks = nn.ModuleList([
#     TransformerBlock(d_model, h, d_ff, dropout)
#     for _ in range(N_LAYERS)])
# x = self.embedding(input_ids)
# for block in self.transformer_blocks:
#     x = block(x, mask=my_mask)
```

■ ■ ■ We Have a 'TransformerBlock', What's next?

- The original Transformer was designed for sequence-to-sequence tasks (like translation).
- To do this, it proposed an architecture with two distinct stacks, each using a specialized version of our block:
- An **Encoder** Stack: to “understand” the input sequence.
- A **Decoder** Stack: to “generate” the output sequence.

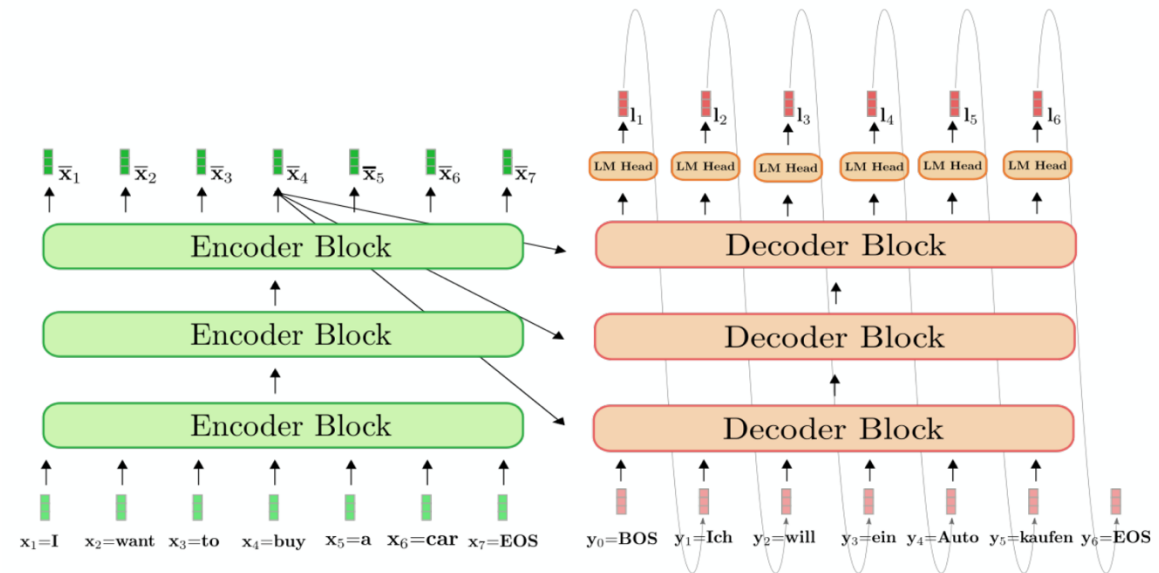
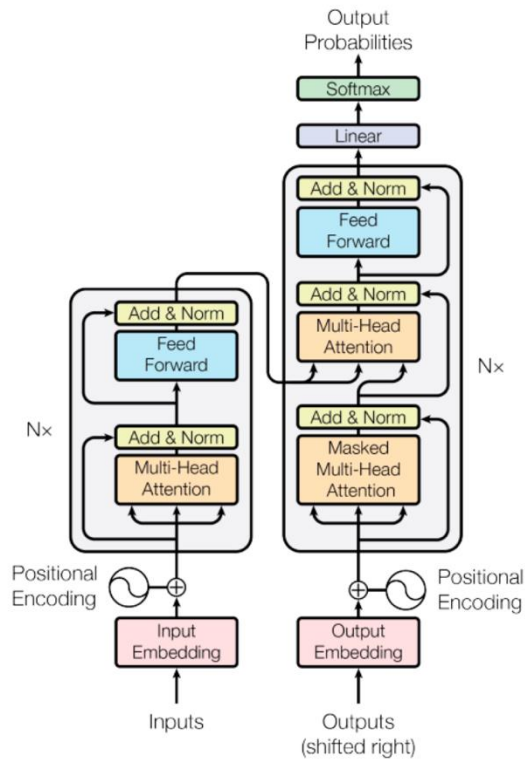
Encoder vs. Decoder Stacks

Encoder Stack: with Multi-Head Attention layers and Feed-Forward layers.

- **Bidirectional**: Each position can see all other positions

Decoder Stack: with **Masked** MHA, **Cross-Attention**, and Feed-Forward layers.

- **Autoregressive**: Each position can only see previous positions



Decoder: Masked Multi-Head Attention

Purpose: The “mask” is essential for autoregressive language modeling. It ensures that the prediction for token t can only depend on previous tokens $1 \dots t - 1$

How it works

- It modifies the attention score matrix before the softmax step.
- A “mask matrix” (with values of $-\infty$) is added to the scores
- After softmax, all “future” positions ($-\infty$) will have a probability of 0.

$$S_{i_masked} = S_i + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Casual (Look-Ahead) Mask in PyTorch

```
import torch
# 1. Create an upper-triangular matrix of 1s (True)
# torch.triu(..., diagonal=1) -> Upper triangle, k=1
# [[0., 1., 1., 1., 1.], [0., 0., 1., 1., 1.],
#  [0., 0., 0., 1., 1.], [0., 0., 0., 0., 1.],
#  [0., 0., 0., 0., 0.]]
causal_mask_bool = torch.triu(torch.ones(seq_len, seq_len), diagonal
                               =1).bool()
print("Causal Mask (True where we fill):\n", causal_mask_bool)

# 2. Add Batch and Head dimensions for broadcasting
final_mask = causal_mask_bool.unsqueeze(0).unsqueeze(0)
print("\nFinal mask shape for SDPA:", final_mask.shape)

# 3. Test in SDPA function
dummy_q = torch.randn(1, 1, seq_len, 64)
dummy_k = torch.randn(1, 1, seq_len, 64)
dummy_v = torch.randn(1, 1, seq_len, 64)
# Pass the mask where True means 'fill'
output, weights = scaled_dot_product_attention(dummy_q, dummy_k,
                                                dummy_v, mask=final_mask)
```

Decoder: Cross-Attention

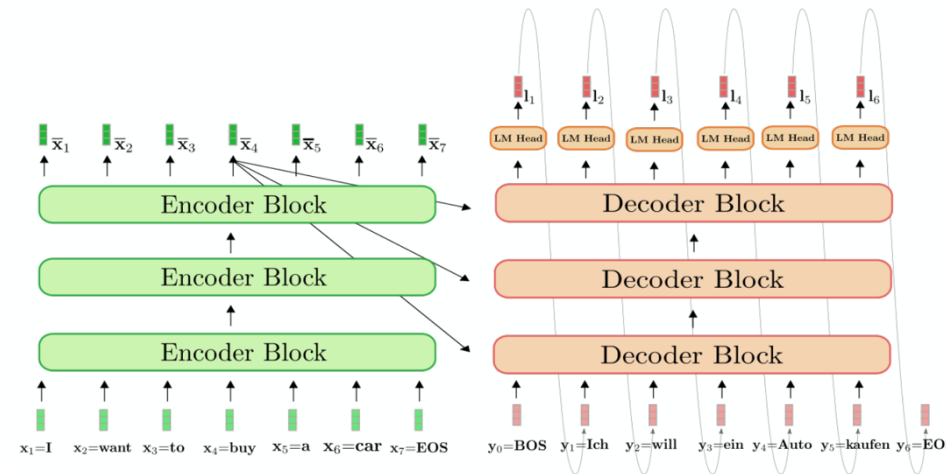
This is the way the Decoder gets information from the Encoder.
It allows the generating (Decoder) stack to “look at” the input (Encoder) stack.

Self-Attention (what we've seen):

- Q , K , V all come from the **same source** (the previous layer's output).
- 'output = mha(x, x, x)'
- "How should tokens in X attend to **other tokens in X** ?"

Cross-Attention (new):

- Q comes from the **Decoder**.
- K and V come from the **Encoder**.
- 'output = mha(x_decoder, encoder_output, encoder_output)'
- "How should Decoder tokens attend to **Encoder tokens**?"



Decoder in PyTorch

```
class DecoderBlock(nn.Module):
    """
    Implements one full Decoder Block
    (Masked MHA -> Add&Norm -> Cross MHA -> Add&Norm -> FFN -> Add&
     Norm)
    """
    def __init__(self, d_model, h, d_ff, dropout_p):
        super().__init__()
        # 1. Masked Multi-Head Attention (Self-Attention)
        self.masked_mha = MultiHeadAttention(d_model, h)
        # 2. Cross-Attention (MHA)
        self.cross_mha = MultiHeadAttention(d_model, h)
        # 3. Feed-Forward
        self.ffn = FeedForward(d_model, d_ff)

        # 4. Three ResidualConnection wrappers
        self.res_conn_1 = ResidualConnection(self.masked_mha, d_model,
                                                dropout_p)
        self.res_conn_2 = ResidualConnection(self.cross_mha, d_model,
                                                dropout_p)
        self.res_conn_3 = ResidualConnection(self.ffn, d_model,
                                                dropout_p)
```

Decoder in PyTorch

```
def forward(self, x, encoder_output, causal_mask, padding_mask=
    None):
    # x shape: [B, Target_T, D], encoder_output shape: [B,
    #   Source_T, D]

    # 1. Apply Masked MHA block (Self-Attention)
    # Q, K, V all come from x
    # We must use the causal (look-ahead) mask
    x = self.res_conn_1(
        x, x_k=x, x_v=x, mask=causal_mask
    )
    # 2. Apply Cross-Attention block
    # Q comes from x (decoder)
    # K, V come from encoder_output
    x = self.res_conn_2(x, x_k=encoder_output, x_v=encoder_output,
        mask=padding_mask # (Masks <pad> tokens in encoder)
    )

    # 3. Apply FFN block
    x = self.res_conn_3(x)
    return x
```

Decoder: Final Output Distribution (GPT-style)

After N decoder layers, we have a final set of token embeddings $X \in \mathbb{R}^{t \times d}$.

How to predict the next token?

- GPT selects only the embedding of the last position, x_t .
- It computes “logits” by projecting this vector into the vocabulary space using the initial embedding matrix M

$$\mathbf{z}_t = \mathbf{x}_t^\top \mathbf{M} \in \mathbb{R}^{|\mathcal{Y}|}$$

- The logits $\mathbf{z}_t$ are converted to a probability distribution using the Softmax function.

$$P(y|context) = \frac{\exp(\mathbf{z}_t[y])}{\sum_{y' \in \mathcal{Y}} \exp(\mathbf{z}_t[y'])}$$

Decoder Final Output in PyTorch

```
class GPTModel(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, h, ...):
        super().__init__()
        # 1. Token Embedding (Matrix M)
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        # 2. Stack of N Decoder-Only Blocks
        # (Note: A GPT block is a DecoderBlock *without* the Cross-
        #       Attention sub-layer)
        self.blocks = nn.ModuleList([
            GPTDecoderBlock(d_model, h, ...)
            for _ in range(n_layers)
        ])
        # 3. Final Layer Normalization
        self.final_norm = nn.LayerNorm(d_model)
        # 4. Final Linear Layer (Projection to Vocab)
        self.final_linear = nn.Linear(d_model, vocab_size)
        # 5. !!! WEIGHT TYING !!!
        # The weights of the embedding layer (M) and the final linear
        # layer are shared.
        self.final_linear.weight = self.token_embedding.weight
```

Decoder Final Output in PyTorch

```
def forward(self, input_ids, targets=None):
    # input_ids: [Batch, Seq_Len]
    # 1. Get Token Embeddings, x shape: [B, T, D]
    x = self.token_embedding(input_ids)
    # 2. Pass through all Decoder Blocks
    for block in self.blocks:
        x = block(x)
    # 3. Final Norm
    x = self.final_norm(x)
    # 4. Project to Vocab Space (Logits)
    logits = self.final_linear(x)
    # logits shape: [B, T, Vocab_Size]
    # 5. Calculate Loss (During Training)
    if targets is not None:
        # Flatten for CrossEntropyLoss
        # (B, T, V) -> (B*T, V); (B, T) -> (B*T)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)),
                               targets.view(-1))
    else:
        loss = None
    return logits, loss
```

Full 'Transformer'

Encoder

```
# (We assume EncoderBlock is already defined)

class Encoder(nn.Module):
    def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
        super().__init__()
        # Create N identical EncoderBlocks
        self.layers = nn.ModuleList([
            EncoderBlock(d_model, h, d_ff, dropout_p)
            for _ in range(n_layers)
        ])
        # Final normalization
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        # x shape: [B, T_src, D]
        # Pass input through all N layers
        for layer in self.layers:
            x = layer(x, mask=mask)

        # Final normalization
        return self.norm(x)
```

Decoder

```
# (We assume DecoderBlock is already defined)
class Decoder(nn.Module):
    def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
        super().__init__()
        # Create N identical DecoderBlocks
        self.layers = nn.ModuleList([
            DecoderBlock(d_model, h, d_ff, dropout_p)
            for _ in range(n_layers)
        ])
        # Final normalization
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, enc_output, causal_mask, padding_mask):
        # x shape: [B, T_tgt, D]
        # Pass input through all N layers
        for layer in self.layers:
            x = layer(x, encoder_output=enc_output, causal_mask=
                causal_mask, padding_mask=padding_mask)
        # Final normalization
        return self.norm(x)
```

Transformer

```
class Transformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model,
                  n_layers, h, d_ff, dropout_p, max_len):
        super().__init__()
        # --- Embeddings ---
        # (We use nn.Embedding for absolute pos,
        #  but RoPE would be inside MHA)
        self.src_embedding = nn.Embedding(src_vocab_size, d_model)
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, dropout_p,
                                                max_len)

        # --- Encoder & Decoder Stacks ---
        self.encoder = Encoder(n_layers, d_model, h, d_ff, dropout_p)
        self.decoder = Decoder(n_layers, d_model, h, d_ff, dropout_p)

        # --- Final Output Layer ---
        self.final_linear = nn.Linear(d_model, tgt_vocab_size)

        # --- (Optional) Weight Tying ---
        # self.final_linear.weight = self.tgt_embedding.weight
```

Transformer

```
def forward(self, src_ids, tgt_ids, src_mask, tgt_mask):
    # src_ids: [B, T_src]; tgt_ids: [B, T_tgt]
    # src_mask: [B, 1, 1, T_src] (for padding)
    # tgt_mask: [B, 1, T_tgt, T_tgt] (causal + padding)

    # 1. Encode the source sequence
    # src_embed: [B, T_src, D]
    src_embed = self.pos_encoding(self.src_embedding(src_ids))
    # enc_output: [B, T_src, D]
    enc_output = self.encoder(src_embed, mask=src_mask)

    # 2. Decode the target sequence
    # tgt_embed: [B, T_tgt, D]
    tgt_embed = self.pos_encoding(self.tgt_embedding(tgt_ids))
    # dec_output: [B, T_tgt, D]
    dec_output = self.decoder(tgt_embed, enc_output, causal_mask=
                              tgt_mask, padding_mask=src_mask)

    # 3. Final projection to logits
    # logits: [B, T_tgt, tgt_vocab_size]
    logits = self.final_linear(dec_output)
    return logits
```

Why did Transformer win?

- It enabled massive parallelization and scalability.
- It is the architectural foundation for nearly all modern AI models:
 - T5, BART (Encoder-Decoder)
 - BERT (Encoder-only)
 - DeepSeek, GPT, Qwen, Llama (Decoder-only)

The Rise of Decoder-Only Models

- In the current era of LLMs, one paradigm has become the most dominant and powerful for general-purpose AI: **Decoder-Only**.
 - Example: GPT series from OpenAI are simply stacks of Decoder blocks (like we've seen, but without the cross-attention part) trained on the core task of Next-Token Prediction.
- However, many state-of-the-art LLM models are “closed-source” or available only via paid APIs. We cannot inspect their weights or architecture.

This has sparked a massive open-source movement to build powerful, transparent, and accessible models.

Key Open-Source Models:

- DeepSeek (from DeepSeek AI)
- Qwen (from Alibaba)
- Llama (from Meta)
- ...

How can we use these models?



Hugging Face Ecosystem

Hugging Face is a hub for state-of-the-art AI models.

It mainly offers:

- **Models:** a vast repository of pretrained AI models that are readily accessible and highly customizable. The platform is community-driven and allows users to contribute their own models.
- **Datasets:** a library of thousands of datasets that you can use to train, benchmark, and enhance your models.
- **Spaces:** Spaces allows you to deploy and share machine learning applications directly on the Hugging Face website.



Hugging Face

Model Cards in Hugging Face

Model cards are files that accompany each model to provide useful information. You can search for the model card you're looking for on the Models page:

The screenshot displays the Hugging Face website's Models page. At the top, the Hugging Face logo and a search bar are visible. The navigation bar includes links for Models, Datasets, Spaces, Buckets, Docs, Enterprise, Pricing, Log In, and Sign Up. On the left sidebar, there are tabs for Main, Tasks, Libraries, Languages, Licenses, and Other. The Tasks section is expanded, showing categories like Text Generation, Any-to-Any, Image-Text-to-Text, Image-to-Text, Image-to-Image, Text-to-Image, Text-to-Video, and Text-to-Speech. Below this is a Parameters section with a slider ranging from < 1B to > 500B. The Libraries section lists various frameworks like PyTorch, TensorFlow, JAX, Transformers, Diffusers, GGUF, MLX, ML, and Transformers.js. The main content area shows a list of model cards with filters for 'Base only' and 'Inference Available', and a sort option for 'Trending'. The model cards listed include:

- tencent/Hy3: Text Generation, 299B, Updated 6 days ago, 8.66k downloads, 710 likes.
- empero-ai/Qwythos-9B-Claude-Mythos-5-1M-GGUF: Image-Text-to-Text, 9B, Updated about 14 hours ago, 1.97M downloads, 2.03k likes.
- zai-org/GLM-5.2: Text Generation, 753B, Updated 10 days ago, 441k downloads, 3.85k likes.
- bottlecapi/ThinkingCap-Qwen3.6-27B: Image-Text-to-Text, 27B, Updated 2 days ago, 4.46k downloads, 247 likes.
- InternScience/Agents-A1: Text Generation, 35B, Updated 4 days ago, 29k downloads, 502 likes.
- froggeric/Qwen-Fixed-Chat-Templates: Updated 9 days ago, 858 likes.
- conradlocke/krea2-identity-edit: Updated 3 days ago, 203 likes.
- baidu/Unlimited-OCR: Image-Text-to-Text, 3B, Updated 9 days ago, 1.43M downloads, 1.94k likes.
- meituan-longcat/LongCat-2.0: Text Generation, 1.8T, Updated 4 days ago, 1.77k downloads, 179 likes.
- HauhauCS/Qwen3.6-35B-A3B-Uncensored-HauhauCS-Aggres...: Image-Text-to-Text, 35B, Updated Apr 17, 2.6M downloads, 2.67k likes.

<https://huggingface.co/models>

Model Cards in Hugging Face

A high-quality model card includes information about the model. In some model cards, you'll also get to tinker with a deployed instance of the model via the Inference API.

The screenshot shows the Hugging Face interface for the model `facebook/bart-large-mnli`. The top navigation bar includes the Hugging Face logo, a search bar, and links for Models, Datasets, Spaces, Buckets, Docs, Enterprise, Pricing, Log In, and Sign Up. The model card header shows the model name, a like count (1.59k), a follow count (13.6k), and various tags including Zero-Shot Classification, Transformers, PyTorch, JAX, Rust, Safetensors, multi_nli, bart, text-classification, and two arXiv links. Below the header, there are tabs for Model card, Files and versions, xet, and Community. The main content area is divided into two columns. The left column contains the model's description: "This is the checkpoint for bart-large after being trained on the MultiNLI (MNL) dataset." and "Additional information about this model:" followed by a list of links: "The bart-large model page", "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension", and "BART fairseq implementation". Below this is a section titled "NLI-based Zero Shot Text Classification" with a detailed paragraph about the model's use as a zero-shot sequence classifier. The right column features a "Downloads last month" graph showing 3,352,801 downloads, a Safetensors section with model size (0.4B params) and tensor type (F32), and an "Inference Providers" section with a "HF Inference API" button. Below the inference providers is a "Zero-Shot Classification" section with a text input field "Text to classify...", a "Possible class names" input field, a checkbox for "Allow multiple true classes", and a "Generate" button. At the bottom right, there are links for "View Code Snippets" and "Maximize".

<https://huggingface.co/facebook/bart-large-mnli>

Hugging Face 'transformers': The Gateway to LLMs

Transformers, a Python library that lets you run just about any model in the Model Hub of Hugging Face.

It provides a simple, unified API to download, load, and use thousands of pre-trained models.

```
(venv) $ python -m pip install transformers
```

■ Hugging Face 'transformers': The Gateway to LLMs

Pipelines are the simplest way to use models out of the box in Transformers

```
from transformers import pipeline
# 1. Load a pre-built pipeline
pipe = pipeline(
    "text-generation",
    model="Qwen/Qwen2-0.5B-Instruct"
)
# 2. Use it!
messages = [
    {"role": "user", "content": "Write a hello world in Python"}
]
output = pipe(messages, max_new_tokens=20)
print(output[0]['generated_text'][-1]['content'])
# --- Output ---
# def hello_world():
#     print("Hello, world!")
```

Hugging Face 'transformers': The Gateway to LLMs

AutoModel Workflow gives you more control over tokenization and generation.

```
from transformers import AutoTokenizer, AutoModelForCausalLM
model_name = "Qwen/Qwen2-0.5B-Instruct"
# 1. Load Tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)
# 2. Load Model
model = AutoModelForCausalLM.from_pretrained(model_name)
# (Add .to("cuda") if you have a GPU)
# 3. Format & Tokenize Prompt
messages = [
    {"role": "user",
     "content": "Tell me a short joke"}
]
text = tokenizer.apply_chat_template(messages, tokenize=False)
inputs = tokenizer(text, return_tensors="pt")
# 4. Generate!
outputs = model.generate(**inputs, max_new_tokens=50)
# 5. Decode and Print
result = tokenizer.decode(outputs[0], skip_special_tokens=True)
print(result)
```

Default: Greedy Search

- ‘model.generate(...)’ by default uses Greedy Search.
- It always picks the single most probable next token.
- It is fast, deterministic, but often “boring” and repetitive

```
# This is greedy search
# (do_sample=False is default)
outputs = model.generate(
    **inputs,
    max_new_tokens=50
)
print(tokenizer.decode(outputs[0]))
```

Use Beam Search & Fix Repetition

- Beam Search explores k (e.g., 'num_beams=4') possible paths at each step.
- It finds the entire sequence with the highest total probability.
- It's often more coherent than Greedy

```
# Task 1: Use 4 beams to find a better joke
beam_outputs = model.generate(
    **inputs,
    max_new_tokens=50,
    num_beams=4,
    early_stopping=True # Stop when all beams finish
)
print(tokenizer.decode(beam_outputs[0], ...)) # Often better!
# Task 2: What if it repeats? "I I I I..."
# Use repetition_penalty to discourage repeated tokens
beam_outputs_fixed = model.generate(
    **inputs,
    max_new_tokens=50,
    num_beams=4,
    early_stopping=True,
    repetition_penalty=1.2 # > 1.0 penalizes repeats
)
print(tokenizer.decode(beam_outputs_fixed[0], ...))
```

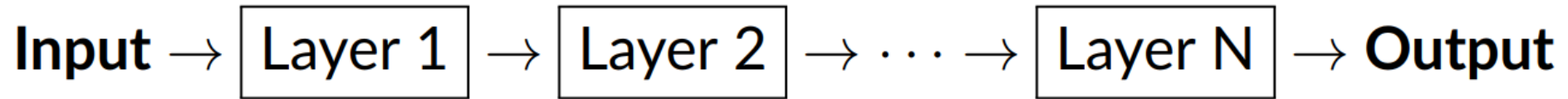
In-Class Practice (Submit to Canvas)

Problem: Sampling introduces randomness into text generation, making outputs more diverse and creative.

- Top-k sampling ($\text{top_k}=50$): randomly samples from the 50 most likely next tokens.
- Top-p sampling ($\text{top_p}=0.9$): randomly samples from the smallest set of tokens whose cumulative probability exceeds 90%.

Task: Compare 'top_k' vs. 'top_p'

- Load a pretrained language model using Hugging Face transformers.
- Use the same set of prompts for all experiments.
- For each prompt,
 - generate at least 3 outputs with $\text{top_k}=50$
 - generate at least 3 outputs with $\text{top_p}=0.9$
- Compare the outputs in terms of diversity, coherence, repetition, and relevance.
- Submit a .ipynb file including your code, generated outputs, and a comparison analysis.



- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN): Handle grid-like data (images)
- Sequence Modeling and RNN: Handle sequential data (text, time series)
- Language Modeling and Transformer: modern AI tasks like text generation

Understanding their design logic helps us customize models for real-world projects.

Choose the right tool for the right task.



Training Strategy

- Regularization
- Normalization
- Weight Initialization & Optimization



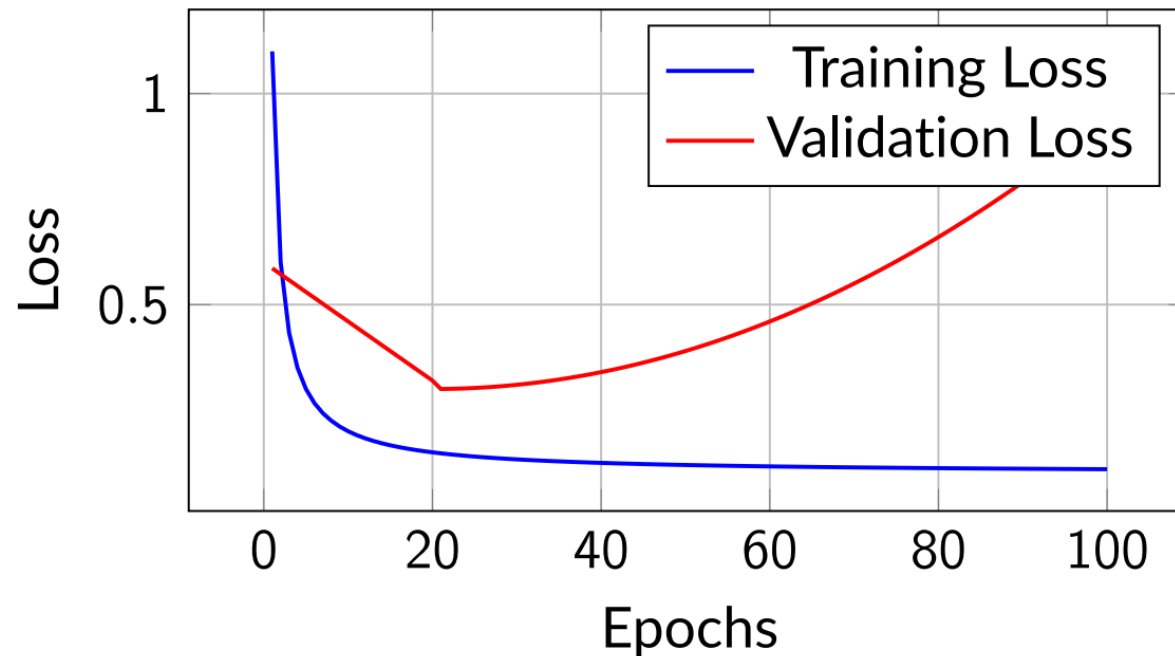
Regularization

The Problem: Overfitting

Symptom: Your model trains perfectly (low training loss) but fails on new data (high test loss).

Diagnosis: The model is too complex. It has "memorized" the training data (including its noise) instead of learning the general pattern.

Cure: Regularization



Regularization = Any technique that constrains the model to make it simpler.

- We want to force the model to find the simplest possible solution that fits the data.
- This is based on "Occam's Razor": the simplest explanation is usually the best.
- A simpler model is less likely to "memorize" noise and more likely to capture the true, general pattern.

- L1 / L2 Regularization (Weight Decay)
- Dropout
- Early Stopping
- Data Augmentation
- Label Smoothing

Regularization: L1 & L2 (Weight Decay)

Idea: "Simple" = "small weights". A model with huge, extreme weights is a sign of overfitting.

How it works: We add a **penalty term** to the loss function.

L2 Regularization (Weight Decay): most common type of regularization, encourages all weights to be small and distributed.

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum w^2$$

In PyTorch: Set `weight_decay=0.01` in the optimizer

Regularization: L1 & L2 (Weight Decay)

Idea: "Simple" = "small weights". A model with huge, extreme weights is a sign of overfitting.

How it works: We add a **penalty term** to the loss function.

L1 Regularization (Lasso): Encourages sparsity (pushes many weights to be exactly 0).

$$L_{\text{new}} = L_{\text{original}} + \lambda \sum |w|$$

In PyTorch: Add an L1 penalty manually to the loss

```
l1_penalty = sum(
    p.abs().sum()
    for name, p in model.named_parameters()
    if "weight" in name
)

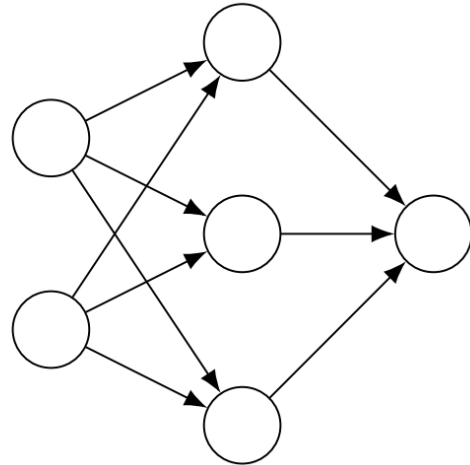
loss = loss + l1_lambda * l1_penalty
```

Regularization: Dropout

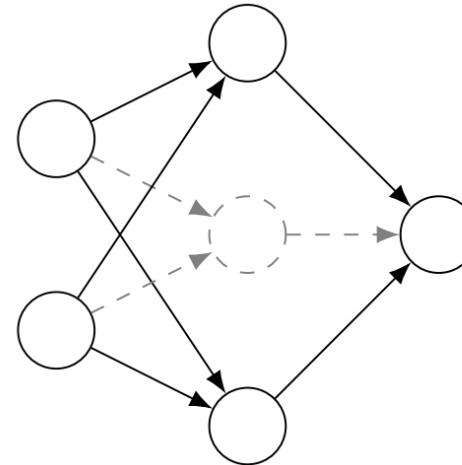
Idea: The model is “cheating” by having neurons co-adapt (e.g., “I’ll only work if Neuron 27 is active”). Prevents “co-adaptation”. Neurons are forced to learn robust, independent features.

How it works: During training, for each forward pass, randomly “drop” (set to 0) a fraction p (e.g., $p = 0.5$) of neurons.

Full Network w/o Dropout



Network w/ Dropout ($p = 0.5$)



Dropout in PyTorch: nn.Dropout

Dropout is only active during training (model.train()).
It is disabled during evaluation (model.eval()).

```
class MLPWithDropout(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim,
                  p_drop=0.5):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
        # Initialize the dropout layer
        self.dropout = nn.Dropout(p=p_drop)
    def forward(self, x):
        x = self.fc1(x)
        x = F.relu(x)
        x = self.dropout(x) # Apply dropout after activation
        x = self.fc2(x)
        x = F.relu(x)
        x = self.dropout(x) # Apply dropout again
        x = self.fc3(x)
        return x
```

Regularization: Early Stopping

The Idea: If overfitting happens over time (as the red line shows), let's just stop training before it gets bad.

How it works:

- Monitor your validation loss (loss on data the model hasn't trained on) at the end of every epoch.
- Keep a copy of the model weights that have the best validation loss so far.
- If the validation loss doesn't improve for N epochs (e.g., $N = 5$, the "patience"), stop training.
- Use the "best" weights you saved, not the final ones

Early Stopping in PyTorch

```
# Early stopping in PyTorch
best_loss = float('inf')
patience_counter = 0
for epoch in range(epochs):
    # ... training code ...
    val_loss = validate_model()
    if val_loss < best_loss:
        best_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), 'best_model.pth')
    else:
        patience_counter += 1
        if patience_counter >= 5:
            model.load_state_dict(torch.load('best_model
            .pth'))
            break
```

Regularization: Data Augmentation

The Idea: If you're overfitting, you don't have enough data. So... Get more data! If you can't collect more, create more by “augmenting” your existing data.

This teaches the model **invariance** (e.g., a “cat” is still a “cat” if it's flipped or in different lighting).

■ Data Augmentation in PyTorch

For Images (using torchvision.transforms)

- Random Flip: Horizontally flip the image.
- Random Crop: Take a random small crop from a large image.
- Random Rotation: Rotate the image by a few degrees.
- Color Jitter: Randomly change brightness, contrast, saturation.

```
import torchvision.transforms as transforms
from torchvision import datasets
import torch
# Define augmentation pipeline
train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),      # Random flip
    transforms.RandomRotation(degrees=15),       # Random rotation
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation
                           =0.2),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
                    0.224, 0.225])
])
# Apply to dataset
train_dataset = datasets.ImageFolder(root='path/to/train/data',
                                     transform=train_transform)
```

Data Augmentation in PyTorch

For Text (using custom transforms)

- Synonym Replacement: Randomly swap words with synonyms.
- Back-translation: Translate (en → fr → en) to get a paraphrased sentence.

```
import random
import nltk
from nltk.corpus import wordnet
from torch.utils.data import Dataset, DataLoader

nltk.download("wordnet")
nltk.download("omw-1.4")

# Define synonym replacement transform
class SynonymReplacement:
    def __init__(self, p=0.2):
        self.p = p # probability of replacing each word

    def get_synonym(self, word):
        synonyms = []

        for syn in wordnet.synsets(word):
            for lemma in syn.lemmas():
                synonym = lemma.name().replace("_", " ")
                if synonym.lower() != word.lower():
                    synonyms.append(synonym)

        if len(synonyms) == 0:
            return word

        return random.choice(synonyms)

    def __call__(self, text):
        words = text.split()
        new_words = []

        for word in words:
            if random.random() < self.p:
                new_words.append(self.get_synonym(word))
            else:
                new_words.append(word)

        return " ".join(new_words)
```

```
# Apply transform in PyTorch Dataset
class TextDataset(Dataset):
    def __init__(self, texts, labels, transform=None):
        self.texts = texts
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = self.texts[idx]

        if self.transform is not None:
            text = self.transform(text)

        label = self.labels[idx]
        return text, label

texts = [
    "The movie was interesting and the actors performed very well.",
    "This product is useful and easy to use."
]

labels = [1, 1]

train_dataset = TextDataset(
    texts=texts,
    labels=labels,
    transform=SynonymReplacement(p=0.2)
)
```

Regularization: Label Smoothing

Idea: Overfitting can happen when the model becomes "over-confident".

- For a “cat” image, the label is $[1, 0, 0]$ (cat, dog, fish). This is a “hard” target. The model is trained to output $p \rightarrow [1.0, 0.0, 0.0]$.
- To get p to 1.0, the logits (inputs to softmax) have to go to $+\infty$. This encourages extreme weights

Label Smoothing: We "smooth" the target labels slightly.

- Instead of $[1, 0, 0]$, we use a “soft” target like:

$$[1 - \epsilon, \epsilon/2, \epsilon/2], \quad \text{e.g., } [0.9, 0.05, 0.05] \text{ for } \epsilon = 0.1$$

- Prevents over-confidence and acts as a regularizer.

In PyTorch: `nn.CrossEntropyLoss` has a `label_smoothing` argument

```
loss_fn = nn.CrossEntropyLoss(label_smoothing=0.1)
```



Normalization

■ Motivation for Normalization

Problem: Internal Covariate Shift

- As the network trains, the weights in Layer 1 change.
- This changes the distribution (mean μ , variance σ^2) of activations that Layer 2 receives.
- From Layer 2's perspective, its input data is constantly shifting.
- This forces Layer 2 to constantly re-adapt, slowing down training.
- It's like trying to hit a moving target.

Solution: Normalization

- Normalize the activations inside the network to keep their mean and variance stable during training.

Normalization: Batch Normalization (BatchNorm)

Idea: Normalize the activations within a mini-batch

For a mini-batch B with m samples.

- Calculate Batch Stats: Find the mean and variance of the activations for each feature/channel across the batch.

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

- Normalize

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

- Scale and Shift: This normalization is too restrictive (forces mean=0, var=1). We let the network learn the best mean/variance.

$$y_i = \gamma \hat{x}_i + \beta$$

Learnable parameters

BatchNorm in PyTorch: nn.BatchNorm2d

Location: Usually applied after Linear/Conv layers and before the activation functions

```
class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.conv = nn.Conv2d(in_channels, out_channels,
                               kernel_size=3, padding=1)

        # BatchNorm layer has learnable parameters (
            gamma, beta),
        # so it must be initialized in __init__
        # We specify the number of *channels* (features)
        self.bn = nn.BatchNorm2d(num_features=
            out_channels)

    def forward(self, x):
        x = self.conv(x)
        x = self.bn(x)    # Apply BN
        x = F.relu(x)     # Apply ReLU
        return x
```

■ Normalization: Layer Normalization (LayerNorm)

Problem with BatchNorm: It depends on the batch size.

- If batch size is 1, variance is undefined.
- If batch sizes are small, the mean and variance are "noisy" estimates.
- This makes it unsuitable for RNNs or Transformers where sequence lengths vary (and batch size is often 1).

Solution: Layer Normalization (LayerNorm)

- Idea: Normalize within a single sample, across its features.
- Stats: Calculate mean and variance **across all features (the layer)** for one sample x_i .
- It is **independent of the batch size**.

LayerNorm in PyTorch: nn.LayerNorm

```
class TransformerBlock(nn.Module):
    def __init__(self, hidden_size):
        super().__init__()
        self.attention = nn.MultiheadAttention(hidden_size, num_heads=8)
        self.ln1 = nn.LayerNorm(hidden_size)
        self.ln2 = nn.LayerNorm(hidden_size)
        self.ffn = nn.Sequential(
            nn.Linear(hidden_size, 4 * hidden_size),
            nn.ReLU(),
            nn.Linear(4 * hidden_size, hidden_size)
        )
    def forward(self, x):
        # x shape: [seq_len, batch_size, hidden_size]
        attn_out, _ = self.attention(x, x, x)
        x = x + attn_out  # Residual connection
        x = self.ln1(x)    # LayerNorm after attention

        ffn_out = self.ffn(x)
        x = x + ffn_out    # Residual connection
        x = self.ln2(x)    # LayerNorm after FFN
        return x
```

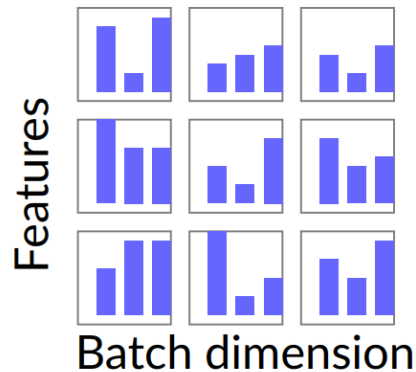
BatchNorm vs. LayerNorm

Batch Normalization

- Normalizes across the batch dimension.
- "For this one channel, what is the avg/stddev across all samples in the batch?"

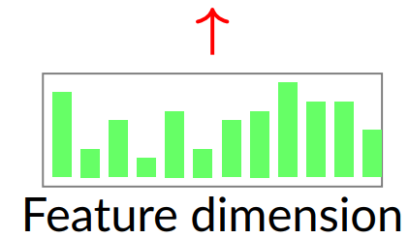
Layer Normalization

- Normalizes across the feature dimension.
- "For this one sample, what is the avg/stddev across all its features?"



↓
Compute mean/std across batch

Compute mean/std across features



How to Choose Your Normalization Layer

For CNNs (Images, Grids): Use BatchNorm

- `nn.BatchNorm1d` for 1D data (MLPs)
- `nn.BatchNorm2d` for 2D data (Images)
- It normalizes across the batch. It assumes:
 - all samples in the batch are i.i.d.
 - the batch size is large enough (e.g., > 16) to get good statistics.

For RNNs/Transformers (Sequences): Use LayerNorm

- `nn.LayerNorm`
- It normalizes across the features of a single sample.
- This is essential for sequential data where:
 - Sequence lengths vary.
 - Batch sizes can be very small (even 1), which would make batch statistics unstable



Weight Initialization & Optimization

Initialization: The “Why”

Problem: How do we set the initial values for weights w ?

What if we set all weights to 0?

- Bad! Every neuron in a layer will be identical.
- They will all get the same input, compute the same output, and receive the same gradient.
- The network will never learn. This is “symmetry breaking”. We must **break symmetry** with random values.

What if we use large random numbers?

- Bad! The inputs to Sigmoid/Tanh will be very large.
- The activations will saturate (be near -1 or 1).
- The gradients will be ≈ 0 (Vanishing Gradients). Training stalls.

Initialization: The "How"

Goal: we need to initialize weights so that the variance (spread) of activations remains roughly the same as we go deeper.

Xavier / Glorot Initialization: For Tanh and Sigmoid activations.

- Samples from a distribution with variance $\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$
- In PyTorch: `nn.init.xavier_uniform_(tensor)`

Initialization: The "How"

Goal: we need to initialize weights so that the variance (spread) of activations remains roughly the same as we go deeper.

He / Kaiming Initialization: For ReLU activations (and its variants).

- ReLU "kills" half the activations, so it needs a different variance.
- Samples from a distribution with variance $\text{Var}(w) = \frac{2}{n_{\text{in}}}$
- In PyTorch: `nn.init.kaiming_normal_(tensor)`
- PyTorch's `nn.Linear` and `nn.Conv2d` layers already use Kaiming He initialization by default. You usually don't have to do this manually.

Whose gradient do we compute?

Batch Gradient Descent

- Compute gradient using the entire dataset.
- Pros: Stable, accurate gradient.
- Cons: Extremely slow. Impossible for large datasets (e.g., ImageNet).

Stochastic Gradient Descent (SGD)

- Compute gradient using one sample at a time.
- Pros: Very fast updates.
- Cons: Very noisy gradient. Loss will be unstable.

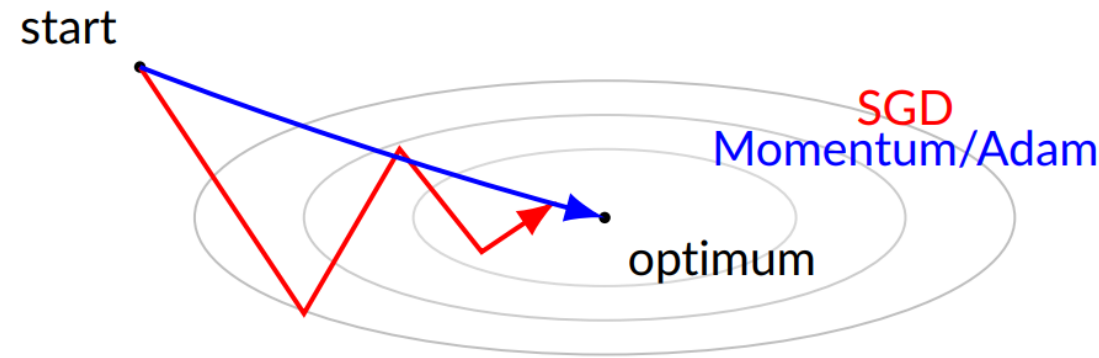
Mini-Batch Gradient Descent (The Standard)

- Compute gradient using a small batch (e.g., 32 or 64 samples).
- **This is what everyone calls "SGD" in practice.**
- Pros: Good balance. Fast, stable, parallelizable (via 'DataLoader')

The Problem with Vanilla SGD

The "vanilla" update rule $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$ has problems:

- **Slow Convergence:** If the loss landscape is a long, narrow "ravine", the gradient will just oscillate side-to-side instead of moving along the bottom.
- **Local Minima:** It can get "stuck" in a small dip (a local minimum) and think it's at the bottom, when the true global minimum is elsewhere.



SGD oscillates (red) while a better optimizer moves directly (blue).

Solution: Use an Optimizer that adapts the learning process

How to Choose Your Optimizer

- SGD with Momentum (SGDM)
- Adagrad
- RMSprop
- Adam
- AdamW

Optimizer: SGD with Momentum (SGDM)

High-level intuition:

- Plain SGD reacts immediately to gradient changes and zig-zags.
- Momentum smooths the direction by keeping part of the previous update, like pushing a heavy object that continues moving despite small bumps.

$$\begin{aligned}g_t &= \nabla_{\theta} L_t, \\v_t &= \beta v_{t-1} - \eta g_t \quad (\text{blend old velocity with new push}), \\ \theta_{t+1} &= \theta_t + v_t \quad (\text{take a step following the velocity}).\end{aligned}$$

$\beta \in [0, 1)$ is the momentum (how much past motion you keep) and η is the step size

```
import torch.optim as optim
optimizer = optim.SGD(model.parameters(), lr=0.01,
                       momentum=0.9)
```

Optimizer: Adagrad

High-level intuition:

- Different parameters need different learning rates.
- Adagrad automatically gives larger steps to rarely-updated parameters and smaller steps to frequently-updated ones, which helps with sparse features.

$$g_t = \nabla_{\theta} L_t,$$

$$r_t = r_{t-1} + g_t^{\odot 2} \quad (\text{ever-growing counter of each coordinate's behavior}),$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{r_t} + \epsilon} \quad (\text{coordinates with lots of action (large } r_t) \text{ slow down}).$$

Note the denominator keeps increasing over time, which can make steps too tiny late in training

```
import torch.optim as optim
optimizer = optim.Adagrad(model.parameters(), lr=0.01,
                           eps=1e-10)
```

High-level intuition:

- Adagrad's steps shrink too fast because it keeps accumulating old gradients.
- RMSprop fixes this by using a moving average, so it adapts to recent gradient magnitudes and keeps learning at a healthy pace.

$$g_t = \nabla_{\theta} L_t,$$

$$s_t = \rho s_{t-1} + (1 - \rho) g_t^{\odot 2} \quad (\text{recent-average of squared slopes}),$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{s_t} + \epsilon} \quad (\text{normalize step: small slopes} \Rightarrow \text{larger steps}).$$

$\rho \in [0, 1)$ controls how fast you forget; ϵ prevents division by zero.

```
import torch.optim as optim
optimizer = optim.RMSprop(model.parameters(), lr=0.001,
                           alpha=0.99, eps=1e-8, momentum=0.0)
```

Optimizer: Adam

High-level intuition: High-level intuition: Adam combine momentum (smooth direction) and RMS-like scaling (safe step size) into one method, giving stable and fast convergence with minimal tuning.

$$g_t = \nabla_{\theta} L_t,$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{direction of travel (momentum)}),$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^{\odot 2} \quad (\text{road roughness (scale)}),$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{fixes small-sample underestimation}),$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad [\text{move along trusted direction with tempered step}].$$

```
import torch.optim as optim
optimizer = optim.Adam(model.parameters(), lr=1e-3,
                        betas=(0.9, 0.999), eps=1e-8)
```

Optimizer: AdamW

High-level intuition: AdamW separates weight decay from the adaptive update, making regularization work properly and improving generalization compared to standard Adam with L2.

$$\theta_{t+\frac{1}{2}} = \theta_t - \eta\lambda\theta_t \quad (\text{decoupled weight decay: a separate small pull toward zero})$$

$$\theta_{t+1} = \theta_{t+\frac{1}{2}} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (\text{regular Adam step on the decayed weights}).$$

Here $(\hat{m}_t, \hat{v}_t)$ are the Adam bias-corrected estimates; λ is the decay strength.

```
import torch.optim as optim
optimizer = optim.AdamW(model.parameters(), lr=1e-3,
                        betas=(0.9, 0.999), eps=1e-8, weight_decay=0.01)
```

How to Choose Your Optimizer

Default Choice (90% of cases): AdamW

- Why: It's fast, converges reliably with minimal tuning, and correctly implements weight decay (L2 regularization), which helps prevent overfitting.
- Start with $lr=1e-3$, $weight_decay=1e-2$.

For Best Performance (SOTA): SGD + Momentum

- Why: If you are willing to carefully tune the learning rate, momentum, and learning rate schedule, SGD+Momentum can often achieve slightly better final validation accuracy than Adam.
- Requires much more manual effort.

Historical / Niche Use:

- Adagrad, RMSprop were important steps toward Adam.
- They are less commonly used today.

Gradient Clipping

Problem: In deep networks (especially RNNs), gradients can become massive, leading to $\nabla_{\theta}L \rightarrow \infty$.

- This "exploding gradient" causes the update $\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta}L$ to be huge.
- The parameters "explode" to NaN (Not a Number).
- Training fails instantly.

Solution: Gradient Clipping

- If the gradient is too big, "clip" it to a maximum value.

Gradient Clipping in PyTorch

After `loss.backward()`, before `optimizer.step()`, we add one line:

```
# --- Inside training loop ---
loss = ...
loss.backward()
# Clip the gradients (rescales all grads if total norm >
    1.0)
torch.nn.utils.clip_grad_norm_(model.parameters(),
    max_norm=1.0)
optimizer.step()
```



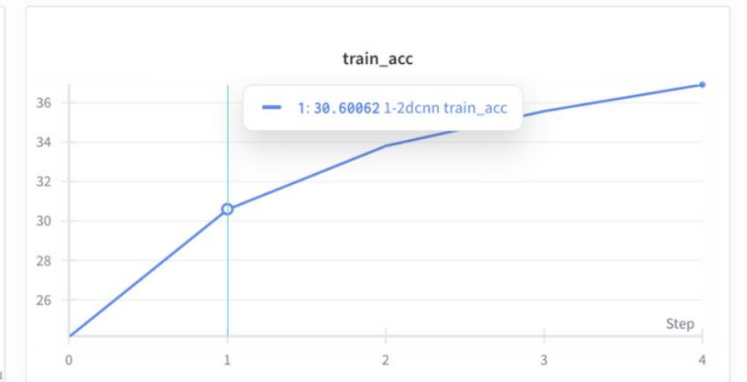
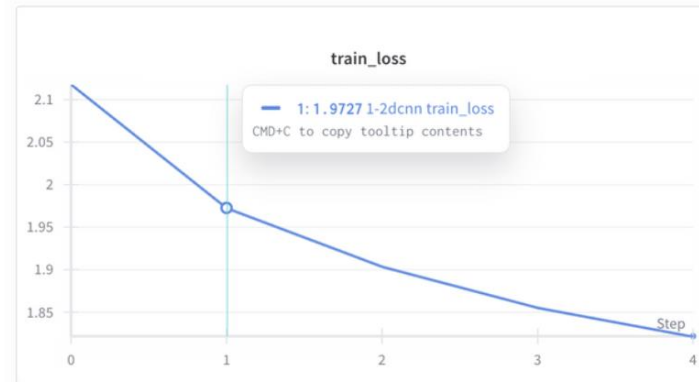
WandB (Weights and Biases)

What is WandB?

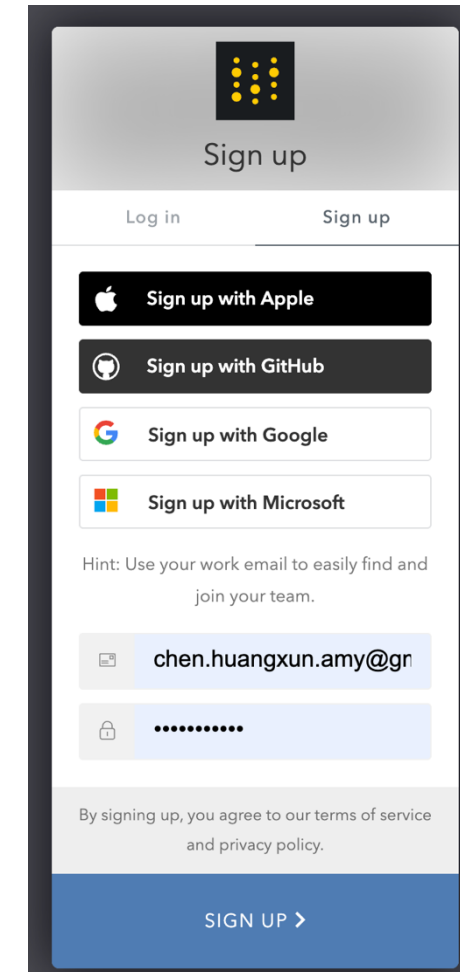
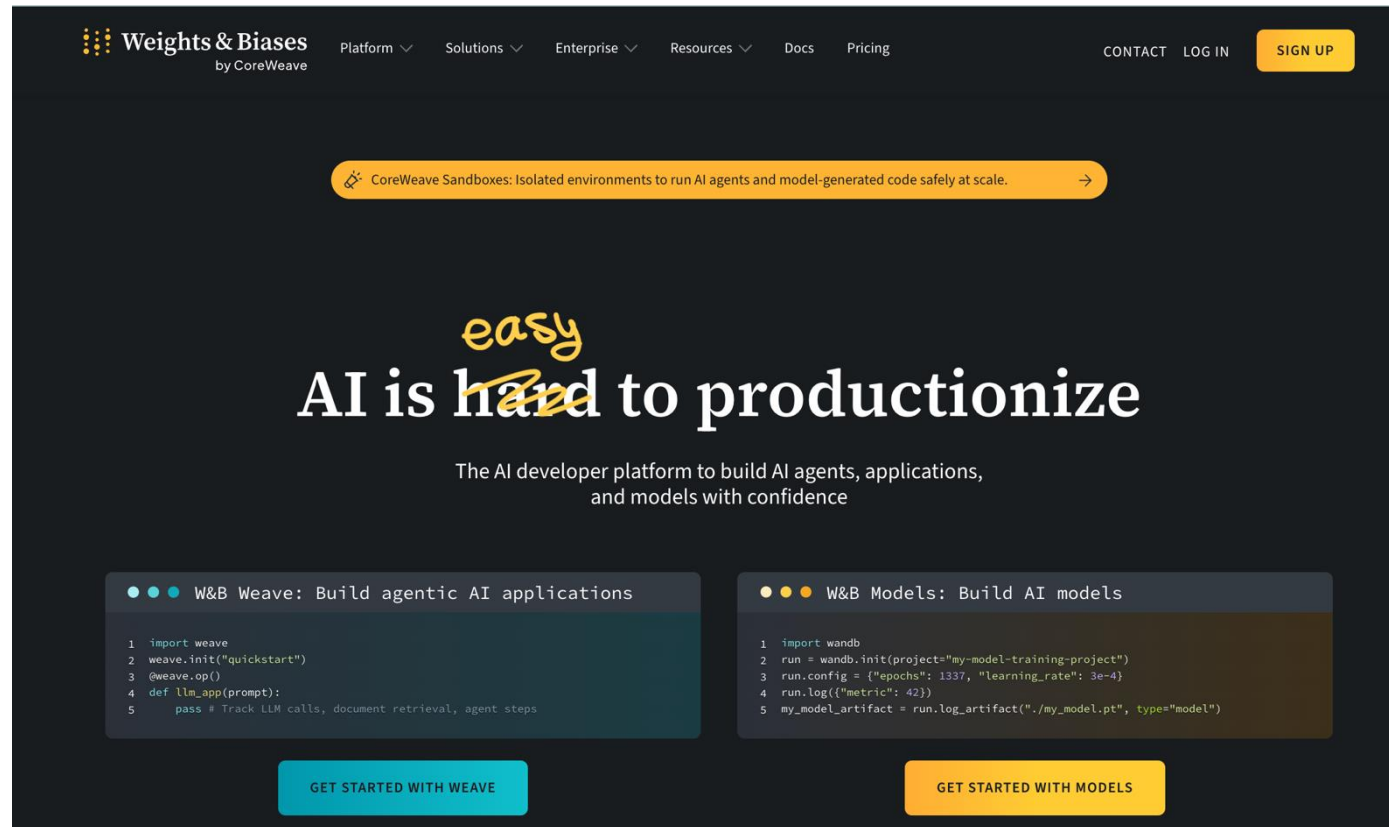
WandB is a powerful ML Operations platform that helps us track, visualize, and manage the experiments seamlessly.

Key Feature:

- Visualize metrics like Loss and Accuracy F1 in real-time as the model trains.
- Keep track of different models and training runs.
- Spot overfitting or exploding gradients instantly using interactive charts.



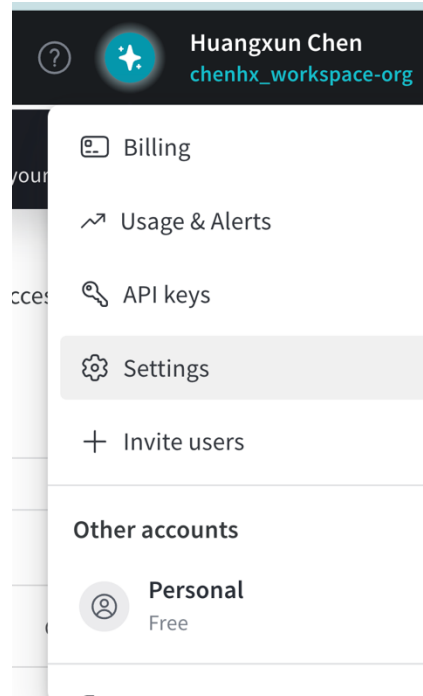
Set Up WandB Account



<https://wandb.ai/site/>

Set Up WandB Account

- Your API key authenticates your machine/notebook so runs can upload to your WandB dashboard.
- You can find your key in the “API Keys” section in your WandB user settings.
- **Never post your API key in GitHub, Piazza, screenshots, or shared notebooks!**
- If it does get leaked, please rotate the key immediately in WandB settings.



API keys

Manage the API keys associated with your **user account**. API keys grant full access to your W&B account and should be kept secure.

2 keys

+ New key

Search by name or API key id

WandB: Installation and Log in

Wandb Installation for Python

```
pip install wandb
```

Login in

```
wandb login
```

paste your API key when prompted.

```
import os, wandb
```

```
os.environ["WANDB_API_KEY"] = "<YOUR_WANDB_API_KEY>"
```

```
wandb.login()
```

Log a Run to a new project

Start tracking system metrics and console logs, right out of the box.
Run this sample code to see the new run appear in W&B.

```
import random

import wandb

# Start a new wandb run to track this script.
run = wandb.init(
    # Set the wandb entity where your project will be logged (generally your team name).
    entity="chenhx_workspace",
    # Set the wandb project where this run will be logged.
    project="my-awesome-project",
    # Track hyperparameters and run metadata.
    config={
        "learning_rate": 0.02,
        "architecture": "CNN",
        "dataset": "CIFAR-100",
        "epochs": 10,
    },
)

# Simulate training.
epochs = 10
offset = random.random() / 5
for epoch in range(2, epochs):
    acc = 1 - 2**-epoch - random.random() / epoch - offset
    loss = 2**-epoch + random.random() / epoch + offset

    # Log metrics to wandb.
    run.log({"acc": acc, "loss": loss})

# Finish the run and upload any remaining data.
run.finish()
```

Deep Model Architecture

- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Sequence Modeling and RNN
- Language Modeling and Transformer
- Library: Hugging Face

Training Strategy

- Regularization
- Normalization
- Weight Initialization & Optimization
- Library: WandB (Weights and Biases)

Python Programming with Artificial Intelligence

Thank you for listening!

